

PROJECT PORTFOLIO

SSketch

함께 그린 그림을 훑치고 평가하는 4인 멀티플레이 게임. Unity의 입력·물리·화면 동기화와 C++ 서버를 연결했다.



프로젝트	팀 프로젝트
담당 역할	Unity 클라이언트 프레임워크, 실시간 입력 및 동기화, C++ IOCP 서버, Vivox 음성 채팅
기술	Unity · C# · C++ · IOCP · TCP · UDP · Vivox
개발 기간	공통 프로젝트 이후 6주 · 기획 3주 / 구현 3주

<https://yollo4179.github.io/projects/ssketch/>

내가 구현한 영역

Unity 클라이언트 프레임워크, 실시간 입력 및 동기화, C++ IOCP 서버, Vivox 음성 채팅

SSketch는 그림을 그리는 시간과 그 그림을 두고 경쟁하는 시간을 하나의 라운드로 묶은 4인 온라인 게임이다. 그림 실력에 더해, 누구의 그림을 가져오고 내 그림을 어떻게 지킬지에 따라 결과가 달라진다. Windows용으로 제작했다.

Unity 클라이언트 프레임워크와 C++ 네트워크 서버를 연결하고, 멀티플레이에서 입력이 처리되는 흐름을 구현했다. 이후 이동이 끊기거나 예측한 점프가 되돌아가는 현상을 입력 생성부터 호스트 판정, 화면 표시까지 나눠 추적했다. Room 채팅과 Vivox 연동도 함께 담당했다.

담당 구현

- **Unity · 클라이언트 프레임워크**
공통 서비스 조립과 앱·씬 수명 관리 · ScriptableObject Catalog, 프리팹 생성과 풀링 · UI Layer 표시 순서·공통 열기와 닫기 · 캐릭터 State, 장비·카메라 Strategy, Animator Adapter · Vivox 방별 음성 채널, 마이크 음소거와 참가자 표시
- **Unity · 입력과 화면**
HostTick / InputTick 기반 입력 예약 · Prediction, Reconciliation, Snapshot Interpolation · 물리 상태와 VisualRoot 화면 보정 분리 · 입력 deadline 및 Jump edge 계측, Pre-Tick Pump
- **C++ · 멀티플레이 기반**
IOCP 기반 TCP·UDP 송수신 · Session, Room, 호스트 권한과 WorldContext 관리 · 입력·스냅샷 검증 및 중계 · Room 작업 직렬화와 도메인 이벤트 전송 연결

기술 구성

Unity · C# · C++ · IOCP · TCP · UDP · Vivox

실행 화면과 구현 코드에 연결된 블로그·영상 링크는 PDF에서 클릭해 열 수 있습니다.

플레이 영상 1 + 기술 상세 12

영상	게임 플레이와 시연	04
01	Unity Host Authority + C++ 서버 구조를 선택한 이유	06
02	Unity 클라이언트와 C++ IOCP 서버 사이의 TCP-UDP 통신 구조	13
03	게임 서버 Room의 책임 분리와 작업 직렬화 구조	19
04	게임 로직과 네트워크를 분리한 이벤트 전달 구조	22
05	HostTick과 InputTick으로 실시간 입력을 어떻게 맞췄는가	26
06	호스트 권위 멀티플레이에서 Prediction-Reconciliation-Interpolation을 분리한 이유	32
07	UDP 패킷은 도착했는데 왜 입력은 누락됐을까? — Host Tick Deadline 추적	41
08	예측에서는 점프했는데 호스트는 왜 거절했을까? — 단발 입력 동기화 문제 해결	49
09	공통 서비스와 씬 객체의 수명을 나눈 클라이언트 구조	56
10	프리팸 조회·생성·재사용의 책임을 나눈 자산 관리 구조	61
11	UI의 표시 순서와 공통 수명을 관리하는 구조	64
12	캐릭터 행동·애니메이션·카메라의 변경 지점을 분리한 구조	71

01 / 플레이 화면

게임 플레이와 시연

낮에는 화가, 밤에는 도둑이 된다. 같은 주제로 그린 그림을 집에 전시하고, 밤에는 다른 플레이어의 그림을 노린다. 아침의 평가가 끝날 때까지 그림의 주인도, 승부도 바뀔 수 있다.

블로그 원문 ↗

SSketch · EC2 시연 영상 · [YouTube에서 보기](#)



실제 게임 화면 · 낮의 그림 그리기



실제 게임 화면 · 밤의 그림 쟁탈

게임 플레이와 시연 · 계속



실제 게임 화면 · 아침의 익명 평가

02 / Architecture

Unity Host Authority + C++ 서버 구조를 선택한 이유

Unity 물리의 일관성과 개발 범위를 함께 고려했다. 호스트는 물리 판정을, C++ 서버는 방과 권한 관리·중계를 맡았다.

[블로그 원문 ↗](#)

문제 여러 플레이어의 이동·충돌·그림 쟁탈 결과를 일관되게 보여줘야 했다.

원인 각 클라이언트가 자기 물리 결과를 확정하면 서로 다른 판정이 생길 수 있었다.

해결 Unity 호스트에 물리 권위를 두고 C++ 서버에서 Room·권한·중계를 관리했다.

결과 입력 → 권위 시뮬레이션 → Snapshot 흐름을 구성하고 실제 4인 실행으로 검증했다.

역할 Unity 클라이언트 프레임워크와 C++ 서버 사이의 입력·상태 전달 구조를 담당했다.

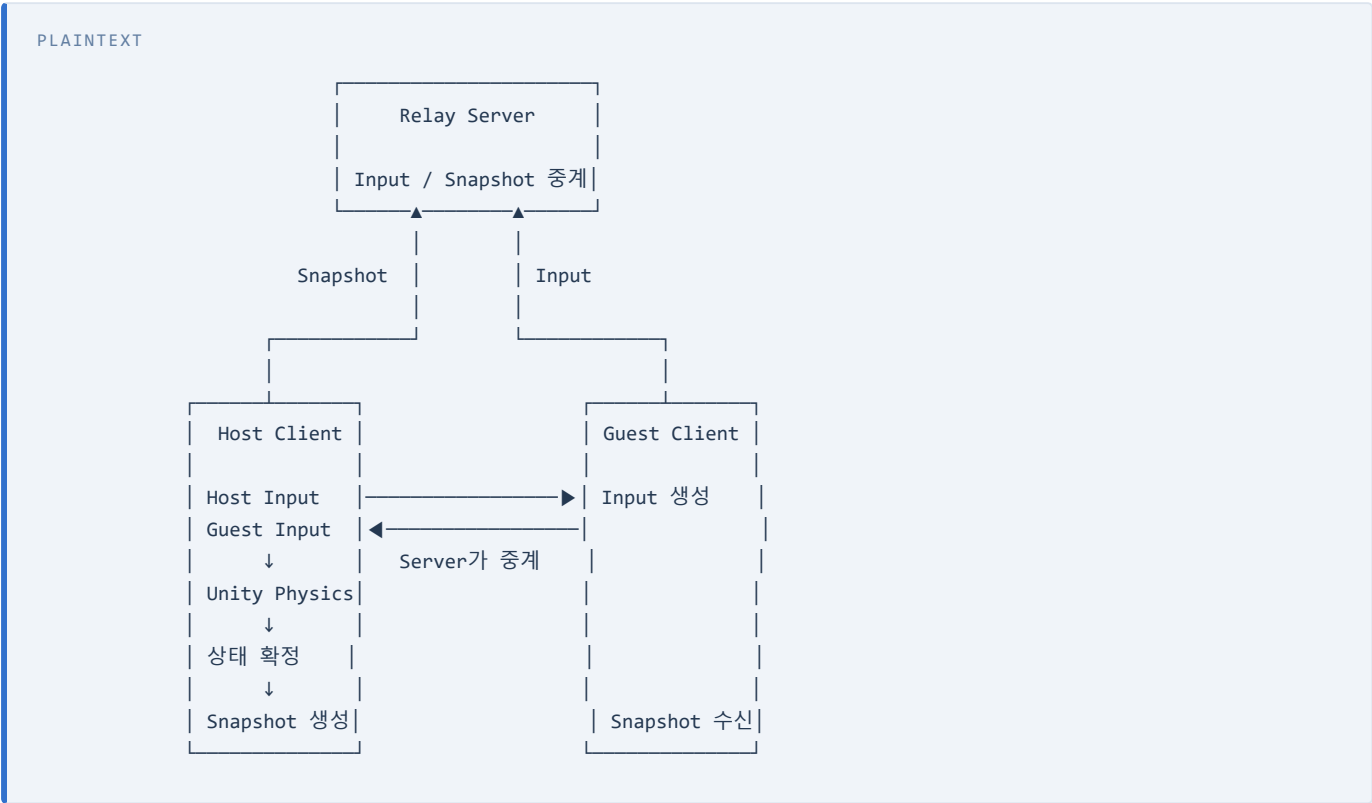
용어 정리

용어	뜻
Host Authority	참가자 중 한 명(방장)의 PC가 물리 판정의 기준이 되는 구조
Guest / Host	Guest는 일반 참가자, Host는 물리 판정을 맡은 방장 PC
Tick	호스트가 물리 계산을 한 번 하는 단위/기준 시간 번호(초당 60번)
Snapshot	호스트가 확정된 전체 상태를 모두에게 보내는 묶음
Room	참가자들이 모여 대기하고 함께 게임을 진행하는 공간
WorldEpoch / AuthorityEpoch	같은 방에서 몇 번째 판인지, 몇 번째 호스트인지 구분하는 문맥 번호
IOCP	Windows에서 여러 소켓의 통신 완료를 한 곳에서 받아 처리하는 비동기 I/O 커널 객체

그림을 빼앗는 게임에 필요한 판정

SSketch는 같은 주제로 그림을 그리고, 밤에는 다른 플레이어의 그림을 가져와 자신의 집에 전시하는 게임이다. 이동만 서로 보여 주는 수준을 넘어, 충돌과 낙백, 상호작용이 여러 화면에서 일관된 결과로 이어져야 했다. 현재 공개 게임은 4인 구성이다.

6주라는 기간 안에 서버에서 모든 클라이언트의 물리 판정을 대신 해 주는 서버 구조를 만들기에는 시간이 부족하다고 판단했다. 물리를 전용 서버에 새로 구현하려면 Unity 물리와 같은 이동·중력·경사·충돌 규칙을 서버에서 재현하고, 그림과 전투 오브젝트를 포함한 월드 상태를 관리하고, 클라이언트 예측과 서버 상태 사이의 재조정 체계까지 따로 만들어야 했다. 그래서 게임에 참여하는 방장이 호스트 역할을 맡아 물리 시뮬레이션을 직접 수행하도록 구성했다.



권위를 둘 위치를 비교했다

후보	얻는 점	감수할 비용
각 Client의 상태를 신뢰	즉시 반응하고 연결이 단순함	충돌 결과 충돌, 상태 조작 검증 부담
Dedicated Authority	플레이어와 독립된 권위 실행	별도 권위 실행 환경·배포·운영 필요
Unity Host Authority	기존 Unity 물리와 게임 코드를 권위 실행에 그대로 사용	호스트 성능·이탈·신뢰에 의존

방에 참여한 Unity 클라이언트 중 방장을 물리 권위 호스트로 지정했다.
호스트는 자신과 게스트 캐릭터를 같은 60Hz 월드에서 물리 시뮬레이션하고, 계산이 끝난 상태를 30Hz 간격으로 스냅샷으로 만들어 모든 게스트에게 보낸다.
C++ 서버는 현재 호스트와 월드 컨텍스트를 검증한 뒤 그 결과를 방 참가자에게 중계한다.

(계산이 끝난 상태 = 플레이어와 오브젝트의 확정된 물리 위치·상태)

호스트가 매 Tick마다 하는 일

호스트(방장)는 다음 순서를 반복한다.

Unity Host Authority + C++ 서버 구조를 선택한 이유 · 계속

1. 입력 수집

이동·점프·조준·발사 입력을 EntityId + InputTick 기준으로 입력 버퍼에 저장한다.

매 HostTick마다 해당 틱의 입력을 꺼내고, 입력이 도착하지 않았다면 짧은 held fallback 또는 neutral fallback으로 틱을 계속 진행한다.

2. 캐릭터 물리 시뮬레이션

자신과 모든 게스트 캐릭터의 입력을 CharacterMotor에 적용한 뒤 Physics.Simulate()를 한 번 실행해 이동·점프·중력·접지·충돌·낙백 결과를 확정한다. Physics.Simulate()는 FixedUpdate() 직후 실행된다. 부하로 인해 이전 호출과 현재 호출 사이 간격이 커지면 FixedUpdate → Physics.Simulate가 한 프레임에 여러 번 몰아서 실행될 수 있다.

3. 캐릭터 상태 캡처

물리 처리가 끝나면 플레이어마다 확정 위치·몸 방향, 이동·외력 속도, 접지·점프·낙하 상태, 기절·순간이동 상태, 그림 윤반 여부, 마지막으로 확정된 InputTick을 캡처한다.

4. 월드 스냅샷 생성 및 전송

60Hz로 물리를 실행하는 동안 2틱마다 한 번씩 30Hz 월드 스냅샷을 만든다. 스냅샷에는 호스트·게스트 플레이어 전원과 바닥에 놓인 그림의 위치·회전·선속도·각속도가 들어간다.

5. 공격 결과 확정

발사 입력이 들어오면 장비 상태·조준 여부·쿨다운을 검사하고 공격 결과를 결정한다.

- HostTick: 물리 시뮬레이션 기준 시점(실제 게임 시간)
- InputTick: 원하는 시점의 HostTick을 예약하기 위한 목표 시점
- held fallback : 입력이 없으면 직전 입력이나 마지막 확정 상태를 계속 사용합니다.(back fill)
- neutral fallback : 입력이 없으면 직전 상태도 이어받지 않고 미리 정한 중립값을 새 입력처럼 사용합니다.(입력 없음)

무엇을 동기화하고 무엇을 이벤트로 처리했나

무기 판정까지 모두 같은 방식으로 옮기지는 않았다.

호스트가 매 틱 계산하는 값은 스냅샷으로 흘려보내고, 한 순간에 결정되는 값은 이벤트로 한 번만 전달하는 것으로 나눴다.

Unity Host Authority + C++ 서버 구조를 선택한 이유 · 계속

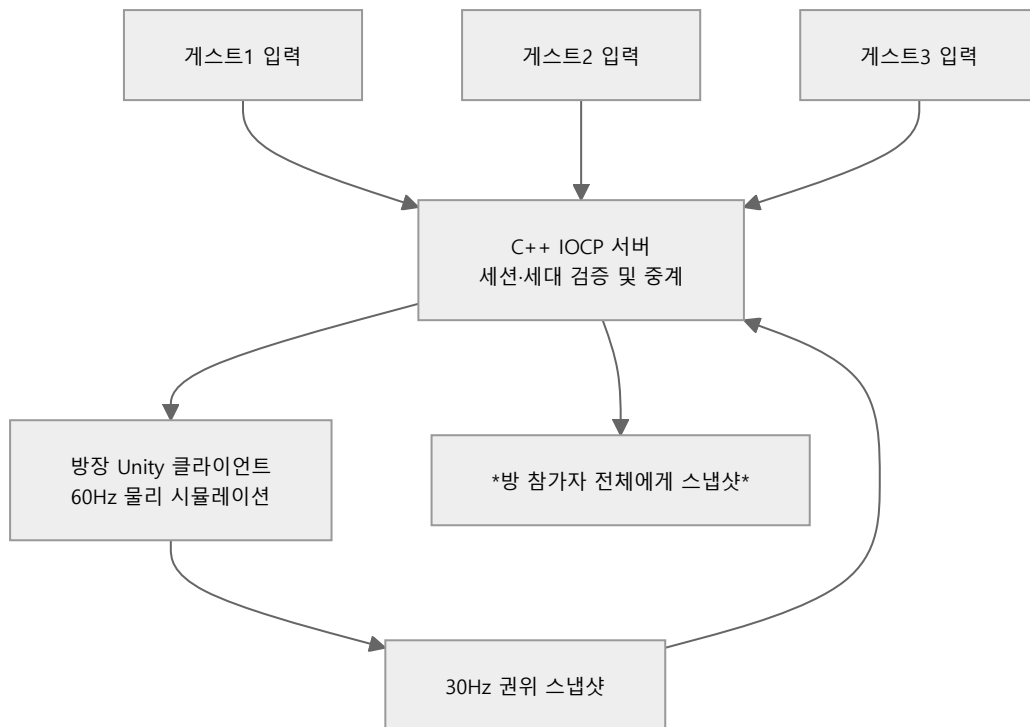
대상·기능	처리 방식	판정 주체
플레이어 위치와 그림(Transform/Rigidbody)	호스트가 계산한 상태를 30Hz 스냅샷으로 전송	호스트
권총·스턴건 같은 히트스캔	탄환 오브젝트를 만들지 않고, 호스트의 즉시 Raycast 결과만 이벤트로 전송	호스트
로켓런처·섬광탄·연막탄의 화면용 투사체	각 클라이언트가 발사 정보를 받아 로컬에서 투사체를 이동·폭발시킴	각 클라이언트
로켓런처의 낙백·기절	호스트가 거리÷탄속만큼 대기한 뒤 미리 정한 위치에서 낙백·기절을 확정	호스트
캐릭터 낙백·기절	클라이언트는 판정하지 않고 호스트가 최종 확정	호스트
섬광탄에 맞은 플레이어	호스트가 피격 대상을 확정하지 않음. 클라이언트마다 폭발 위치와 결과가 달라질 수 있음	각 클라이언트

Unity 호스트와 C++ 서버의 역할 분리

구분	Unity 호스트	C++ 서버
담당 권한	시뮬레이션 권한	세션·중계 권한
책임	입력 선택, 캐릭터 이동·중력·점프·충돌, 최종 위치 계산, Snapshot 생성	현재 호스트 확인, 세션과 Entity 연결, 월드 세대 검증, UDP 중복 제거, 입력·Snapshot 중계

중앙 서버는 호스트가 보낸 좌표의 형식과 범위는 검사하지만, 그 좌표가 물리적으로 가능한지 다시 시뮬레이션하지 않는다. 게임 물리의 원본은 Unity 호스트이고, 그 원본을 안전하게 유통하는 주체는 C++ 서버라는 뜻이다.

Unity Host Authority + C++ 서버 구조를 선택한 이유 · 계속

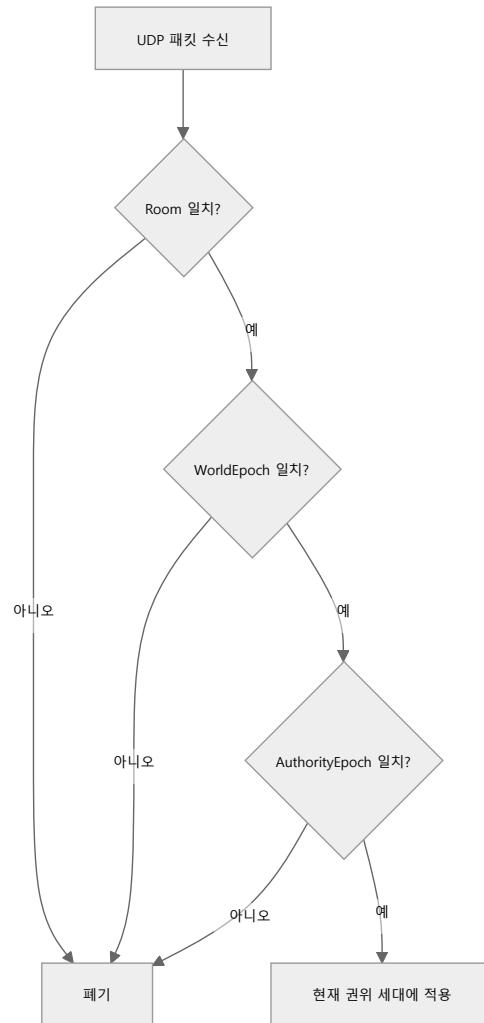


이전 월드의 패킷을 막는 권위 경계

모든 입력과 스냅샷은 다음 세 값으로 구성된 `WorldContext`를 포함한다.

필드	의미
<code>RoomIdentifier</code>	어느 방의 패킷인지 식별
<code>WorldEpoch</code>	같은 방에서 몇 번째 월드 실행인지 식별
<code>AuthorityEpoch</code>	현재 권위 호스트가 몇 번째인지 식별

방 번호만 검사하면 이전 게임의 낡은 UDP 패킷이나 교체 전 호스트의 패킷이 새 게임에 적용될 수 있다. 세 값을 함께 검사해야 이런 패킷을 걸러낼 수 있다.



방 내부 상태를 변경하는 작업은 Room Job Queue로 직렬화했다. 동시에 여러 I/O 완료가 들어와도 한 Room의 변경은 순서대로 실행된다. IOCP·Room 작업 큐의 세부 구조는 [TCP·UDP 통신 글](#)에서 다뤘다.

선택의 비용은 실제 플레이에서 드러났다

Host Authority는 Unity 물리를 한 곳에서 실행하는 장점이 있지만, 호스트가 멈추면 권위 진행도 영향을 받는다. Guest가 자신의 움직임을 예측하고 있어도 확정 결과가 늦어지는 문제까지 없앨 수는 없다.

호스트 이탈도 고려해야 한다. 지금은 진행 중 호스트 이탈을 게임 종료 경로로 처리한다. (호스트의 연결이 끊겼다는 Sysytem UI를 띄운다.)

호스트가 신뢰 경계 안에 들어오는 비용도 남는다. 중앙 서버가 Session과 권한을 확인하더라도, 호스트가 만든 물리 결과의 정당성을 독립적으로 증명하려면 별도의 검증이나 권위 실행이 필요하다.

구조를 만든 뒤에 확인한 문제들

로그를 파일로 찍어보고 ipynb파일로 집계해보니, 4인 실행에서는 패킷이 호스트까지 도착해도 Resolve 전에 입력 버퍼에 들어가지 않아 fallback이 생겼다.

Pre-Tick Pump로 실행 순서를 수정한 수동 비교에서 fallback은 21.95~24.14%에서 3.09~4.57%로 낮아진 것을 확인했다.

Unity Host Authority + C++ 서버 구조를 선택한 이유 · 계속

또한 Guest가 예측한 점프가 호스트에서 거절되는 문제와, 물리 위치를 그대로 렌더링할 때의 화면 캐릭터가 불연속적으로 움직이는 이상현상을 만났다. 이 문제들은 전체 구조 그림만으로 드러나지 않았고, 시간축 / 입력 deadline / 상태 보정 / 화면 보간을 나눠 구현하고 측정해야 했다.

이어지는 글은 [HostTick](#)과 [InputTick](#)부터 시작한다. 입력을 언제 실행할지 정한 뒤, 실제로 deadline을 놓친 경로와 단발 입력을 추적하고, 최종 상태를 화면에 자연스럽게 표시한 과정으로 이어진다.

03 / Architecture

Unity 클라이언트와 C++ IOCP 서버 사이의 TCP·UDP 통신 구조

소켓 완료 통지에서 Room 작업과 Unity 동기화까지 연결했다. TCP 스트림 조립, UDP 검증, 버퍼 수명과 송신 완료의 경계를 정리했다.

[블로그 원문 ↗](#)

문제 Unity 입력·Snapshot 중계와 Room 제어를 동시에 처리할 통신 기반이 필요했다.

원인 메시지마다 전달 특성과 검증 대상이 다르고, 여러 I/O 작업이 동시에 완료될 수 있었다.

해결 IOCP 위에 TCP 스트림 처리와 UDP datagram 경로를 두고 Room 변경을 직렬화했다.

결과 실제 게임 연결과 별도로 최대 200개 합성 클라이언트 조건의 지연·수신 수를 기록했다.

역할 C++ 송수신·Room 연계와 Unity 메시지 처리 흐름을 구성했다.

용어 정리

용어	뜻
IOCP	Windows에서 여러 소켓의 통신 완료를 한 곳에서 받아 처리하는 기술
TCP / UDP	순서를 보장하며 느린 통신 프로토콜 / 순서를 보장 안 하지만 빠른 통신 프로토콜
Session	서버가 클라이언트 한 명과 유지하는 연결 단위
Room	참가자들이 모여 대기하고 함께 게임을 진행하는 공간
datagram	UDP로 한 번에 보내는 패킷 한 덩어리
worker	통신 완료 처리를 실제로 수행하는 스레드
Packet ID	패킷이 어떤 종류인지 구분하는 번호
WorldContext / AuthorityEpoch	어느 방·어느 판·몇 번째 호스트의 메시지인지 구분하는 정보

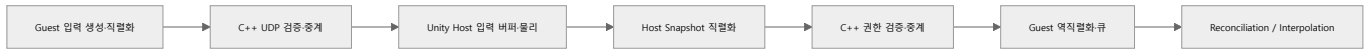
Unity 입력이 호스트까지 가는 기반

Host Authority 구조에서는 Guest의 입력과 호스트의 Snapshot이 계속 오가는 동안 방 입장, 퇴장과 게임 진행도 처리해야 한다. 이 흐름에 필요한 Session·Room·Relay(중계)·GameLoopRule을 직접 구성하기 위해 C++ 서버를 만들었다. Windows 비동기 소켓의 완료 통지는 IOCP로 모았다.

Unity 클라이언트와 C++ IOCP 서버 사이의 TCP-UDP 통신 구조 · 계속

Guest에서 직렬화한 입력은 UDP로 서버에 도착한다. 서버는 발신 Session과 Entity, 현재 Room과 권한 문맥을 확인한 뒤 호스트로 중계한다. 그리고, 서버는 호스트의 확정 상태를 검증해 Guest로 보낸다. Guest가 역직렬화한 데이터는 Reconciliation 또는 Snapshot Interpolation으로 이어진다.

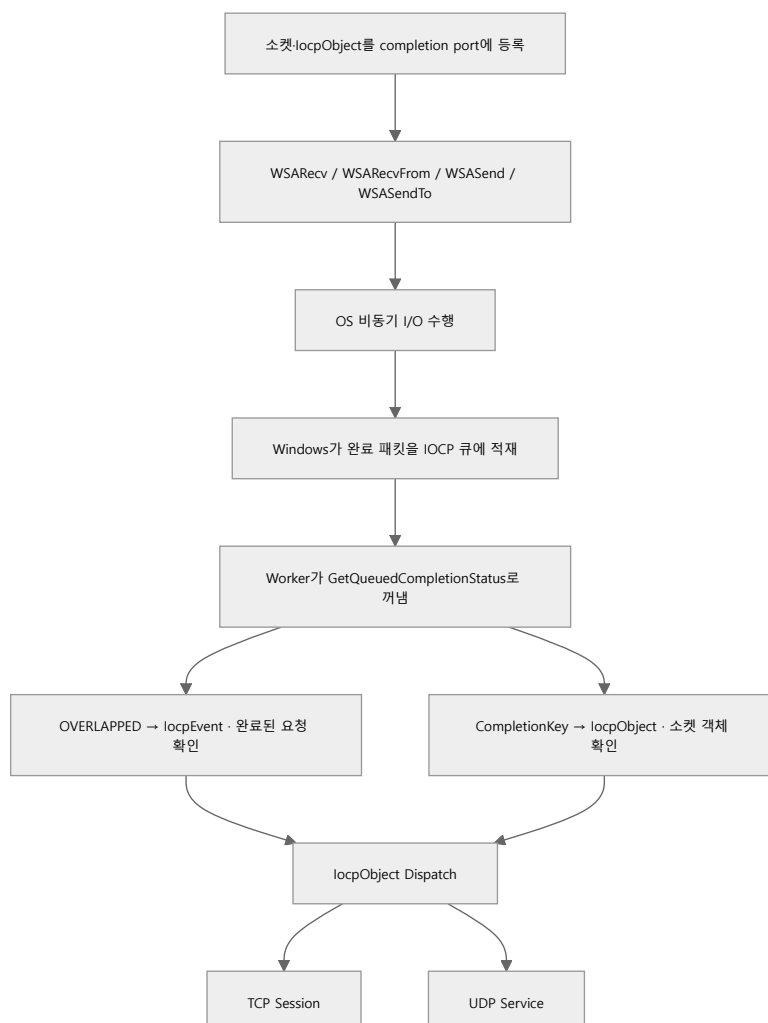
Reconciliation : 최근 확정된 HostTick의 위치 및 상태를 해당 틱 이후의 예측 입력을 적용하는 것
 Snapshot Interpolation: Reconciliation의 결과와 현재 예측 상태의 차이가 많이 나면 Reconciliation값을 타깃으로 천천히 보간하는 것



이 글에서는 큐에서 입력을 꺼내 물리에 적용하는 시점보다, 그 데이터를 운반하는 네트워크 경로에 초점을 맞췄다. 실제 소비 deadline은 [입력 파이프라인 글](#)에서 이어진다.

IOCP-Core와 완료 큐

[IocpCore](#)는 completion port를 만들고 소켓 핸들을 등록한다. 비동기 요청에는 OVERLAPPED를 포함한 이벤트 객체와 버퍼를 연결한다. 이후 OS가 I/O를 완료하면 completion packet을 큐(completion port)에 넣고, [GetQueuedCompletionStatus](#)에서 기다리던 worker가 완료 정보를 받는다.



TCP 수신 시 WSARecv가 RecvBuffer의 현재 쓰기 위치를 직접 가리키도록 구성했다. 수신 완료 후에는 [ProcessRecv](#)를 통해 실제 수신된 바이트 수만큼 버퍼의 쓰기 위치를 갱신하고, 동일한 버퍼에서 완성된 패킷을 순차적으로 해석한다.

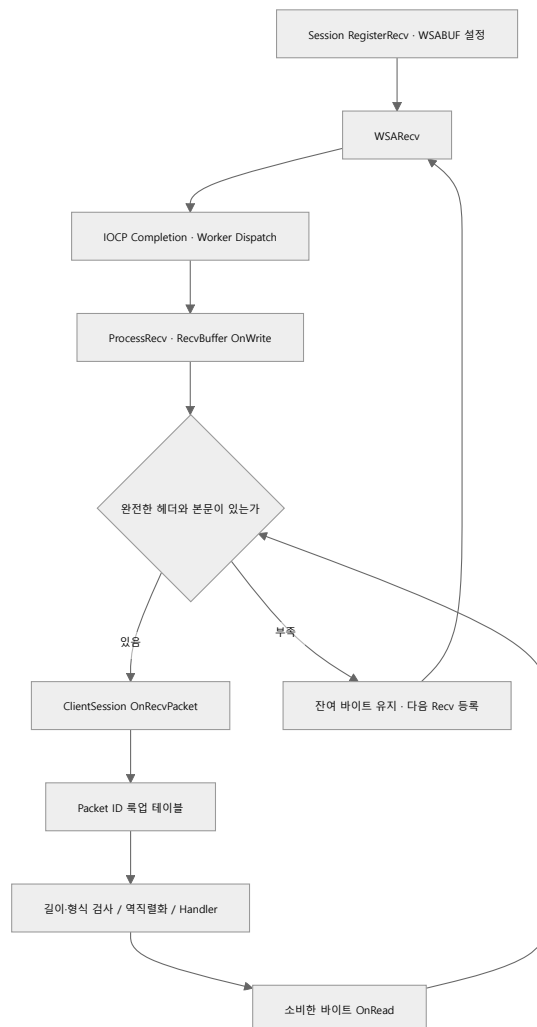
Unity 클라이언트와 C++ IOCP 서버 사이의 TCP-UDP 통신 구조 · 계속

요청한 I/O가 끝나면 Windows는 전송 바이트 수, completion key, OVERLAPPED 포인터를 담은 완료 패킷을 IOCP 큐에 넣는다. 각 IOCP worker 스레드는 `GetQueuedCompletionStatus`를 호출해 큐에서 완료 패킷을 기다린다. 패킷이 도착하면 Windows가 이를 큐에서 꺼내 해당 worker 스레드에 완료 정보를 돌려준다. worker는 completion key로 소켓을 처리하는 `IoctlObject`를 찾고, OVERLAPPED 포인터로 완료된 `IoctlEvent`를 확인한 뒤 해당 객체의 Dispatch에 이벤트를 넘긴다. TCP의 `ProcessRecv`는 완료 바이트 수만큼 기록 위치를 갱신하고 같은 버퍼에서 패킷을 해석한다. UDP 수신 경로도 완료 이벤트를 받은 뒤 해당 수신 버퍼를 처리한다.

completion key에는 처리할 객체를 연결해 두고, 이벤트가 그 객체를 붙잡고 있는 동안은 Session이나 버퍼가 완료되기 전에 먼저 사라지지 않도록 살려 둔다. 함수 호출이 끝나서 반환됐다는 것과, 실제 데이터 송수신이 끝났다는 것은 서로 다른 시점이라는 점이 중요하다.

TCP: 받은 바이트에서 완전한 패킷을 꺼낸다

TCP는 순서 있는 바이트 스트림을 제공한다. 한 번의 `WSARecv` 완료가 한 게임 패킷과 일치하지 않는다. 헤더 일부만 올 수도 있고, 여러 패킷이 합쳐져 도착할 수도 있어 `RecvBuffer`에 받은 바이트를 누적했다.



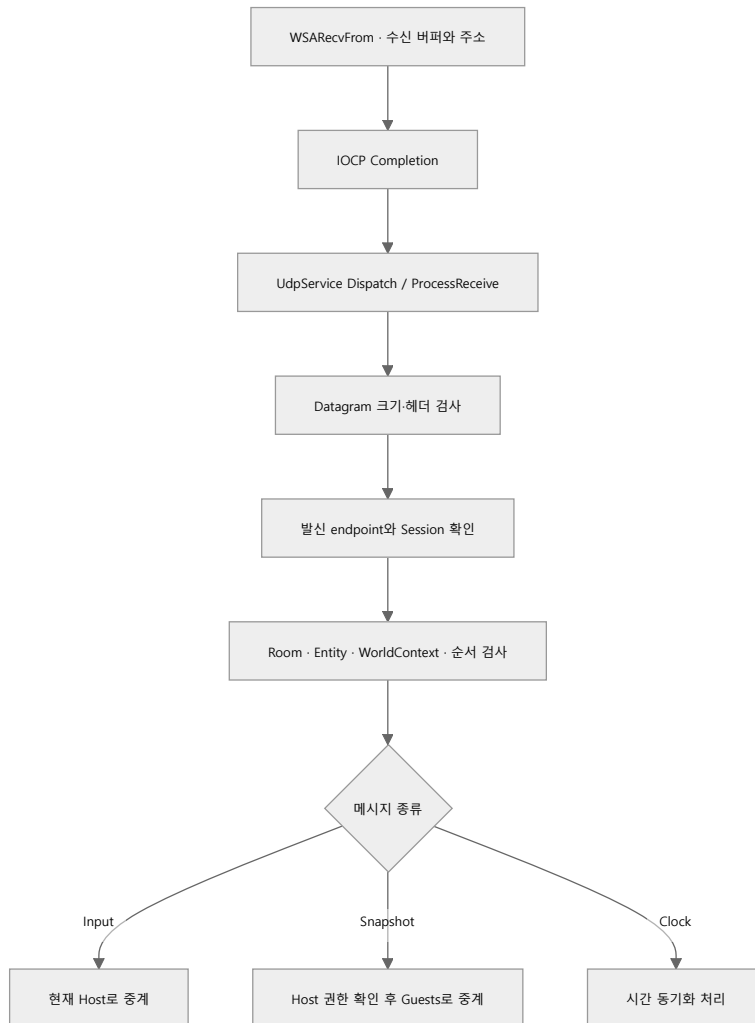
헤더 길이와 패킷 크기를 확인한 뒤 완전한 본문이 있을 때만 handler로 넘겼다. 고정 길이 패킷과 가변 길이 패킷의 wrapper는 각각 필요한 형식을 확인하고 메시지를 해석한다. 록업 테이블은 packet ID에 대응하는 처리 함수를 찾는 역할을 한다.

여기서 바이트 수를 잘못 소비하면 다음 패킷의 헤더 위치까지 틀어진다. 일부만 도착한 패킷은 버퍼에 남기고, 처리한 만큼만 읽기 위치를 이동한 뒤 다음 수신을 등록한다. 소켓의 수신 완료와 애플리케이션 메시지 하나의 완성을 분리한 구조다.

UDP: datagram 하나의 문맥을 확인한다

UDP는 `WSARecvFrom`으로 datagram과 발신 주소를 받는다. TCP처럼 이전 데이터 뒤에 이어 붙여 패킷을 조립하는 대신, 현재 datagram의 크기와 헤더를 먼저 검사한다. 이후 packet ID에 따라 입력, 호스트 Snapshot, 시간 동기화 등의 경로로 나눈다.

`WSARecvFrom`이 datagram을 수신 버퍼에 기록하고, 완료 후 `UdpService::ProcessReceive`가 같은 버퍼의 데이터와 발신 주소를 읽어 검사한다.



입력은 보내는 사용자가 해당 Entity를 제어할 수 있는지 확인하고, Snapshot은 현재 호스트가 보낸 것인지 확인한다. 방과 world/authority epoch가 다르면 이전 게임에서 지연된 datagram이 현재 상태에 섞일 수 있으므로 함께 검증한다.

Room 작업은 한 번에 하나씩 실행한다

여러 IOCP Worker가 같은 Room의 입장·퇴장·게임 진행 상태 변경을 동시에 처리할 수 있기 때문에, Room 상태를 변경하는 작업은 `Room::PushJob()`을 통해 `JobQueue`에 넣어 직렬화했다. 하나의 Room에서는 한 실행자만 Job을 순차적으로 처리하고, 서로 다른 Room의 Job은 각각 독립적으로 실행될 수 있도록 구성했다.

또한 lock은 Job의 추가·추출과 실행자 상태를 보호하는 구간에서만 사용하고, 실제 Job 실행 중에는 잡고 있지 않도록 분리했다. 이를 통해 같은 Room의 상태 변경 순서는 보장하면서도, 서로 다른 Room의 작업까지 하나의 lock으로 직렬화되는 것을 피했다.

Unity 클라이언트와 C++ IOCP 서버 사이의 TCP-UDP 통신 구조 · 계속



도메인 이벤트에서 송신 완료까지

이벤트 버스(발행 구독 패턴 적용)

std.variant와 visit을 활용해 이벤트 객체와 익명함수를 연결하여 호출하도록 하였다.

이벤트를 publish로 전달하면 이벤트를 매개변수로 받는 람다함수들을 조회해서 시그니처가 일치하는지의 여부를 판단해서 매핑한다.

게임 진행 로직이 발생시킨 이벤트는 방에 등록된 이벤트 수신기가 받아 네트워크 알림으로 연결한다. 브로드캐스터는 알림을 패킷으로 만들어 참가자에게 송신을 요청한다. 운영체제가 비동기 송신을 완료하면 IOCP worker가 완료 통지를 처리한다.

TCP에서는 Session의 SendBuffer를 송신 큐에 넣고, 등록 상태를 확인해 WSASend를 요청한다. 완료될 때까지 버퍼를 유지하고 부분 송신이면 남은 offset을 이어 처리한다. 완료 처리 후 다음 송신을 등록한다.

UDP의 WSASendTo도 별도 send event와 버퍼의 수명을 완료 시점까지 유지한다.

TEXT

도메인 상태 변경 → Publish(event) → 어댑터 → Broadcaster
→ Session Send Queue / UDP Send Event → WSASend / WSASendTo
→ OS I/O 완료 → IOCP Worker → 송신 상태·버퍼 정리

송신 완료는 로컬 비동기 I/O의 완료다. 상대 클라이언트가 역직렬화를 끝냈거나 Tick에 적용했다는 응답까지 뜻하지 않는다. 게임 쪽에서 정말 끝났다는 확인은 별도의 확정 신호나 상태 메시지로 따로 정의해야 한다. 수신에서도 송신에서도 이 경계를 유지해야 로그를 잘못 해석하지 않는다.

Python 콘솔 클라이언트로 서버 중계를 확인했다

이 실험의 목적은 Unity 플레이가 아니라, 여러 클라이언트가 접속했을 때 C++ 서버가 게임 프로토콜을 처리하고 Input과 Snapshot을 중계하는지 확인하는 것이었다. Python 콘솔 클라이언트는 TCP-UDP 소켓으로 로그인, 방 생성·입장, 게임 시작 절차를 진행한 뒤 30Hz Input을 보냈다. 각 방의 호스트 역할을 맡은 클라이언트는 4개 캐릭터의 합성 상태를 담은 30Hz Snapshot을 보냈다. 서버는 이 메시지를 실제 소켓에서 수신해 검증하고 방 참가자에게 중계했다.

로컬 계측 PC · 실험 당시와 동일하게 유지한 환경

항목	구성
CPU	Intel Core Ultra 7 155H · 16코어 / 22논리 프로세서
GPU	NVIDIA GeForce RTX 4050 Laptop GPU
메모리	약 32GB · OS 인식 31.51GiB
OS	Windows 11 Enterprise

Unity 클라이언트와 C++ IOCP 서버 사이의 TCP-UDP 통신 구조 · 계속

4·40·100·200클라이언트 실행에서 Input과 Snapshot의 관측 수는 각 조건의 집계 기준과 일치했고, 중계 누락은 집계되지 않았다. 따라서 짧은 합성 부하에서 소켓 연결과 서버 중계 경로가 동작한 것은 확인했다.

게임 서버 Room의 책임 분리와 작업 직렬화 구조

방 상태는 Room에 모으고 게임 진행·평가·호스트 권한은 기능별 서비스에 나눴다. 여러 worker와 타이머가 전달한 상태 변경 작업은 방별 큐에서 순서대로 실행한다.

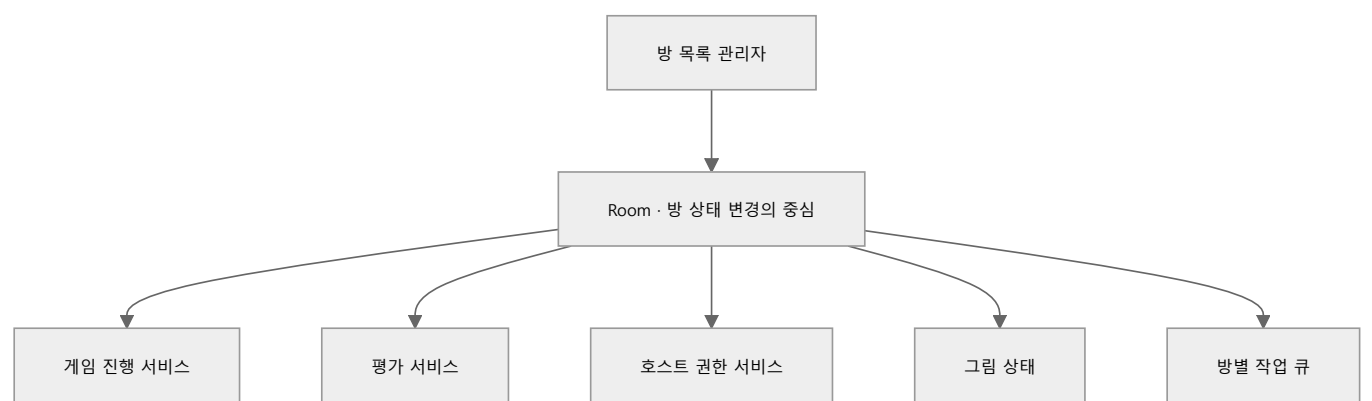
블로그 원문 ↗

Room에 모을 상태와 나눌 책임

Room은 참가자들이 모여 대기하고 함께 게임을 진행하는 공간이다. 참가자와 준비 상태뿐 아니라 게임 시작 이후의 캐릭터·그림 상태도 같은 방에 속한다. 서버는 이 상태들을 Room에서 관리하고, 요청 처리기는 Room의 상태 변경 기능을 호출하도록 구성했다.

방 상태와 게임 진행에는 서로 다른 책임이 있다. Room은 참가자와 방 설정을 관리한다. 게임 진행 서비스는 라운드와 페이지를 전환하고, 평가 서비스는 점수와 결과를 관리한다. 호스트 권한 서비스는 물리 판정을 맡을 호스트와 월드·권한 세대를 관리한다.

구성	책임
RoomManager	방 생성·조회·제거, 방 ID와 입장 코드로 방 찾기, 방 객체 보관
Room	참가자·Ready·Host·게임 상태, 플레이어와 Entity 연결, 캐릭터 상태 관리
GameLoopService	게임 진행과 라운드·페이지 전환
EvaluationService	평가 진행과 점수·결과 관리
HostAuthorityRoomService	호스트 권한과 월드·권한 세대 관리
DrawingState	방에 속한 그림 상태 보관
JobQueue	같은 방의 상태 변경 작업을 순서대로 실행



게임 서버 Room의 책임 분리와 작업 직렬화 구조 · 계속

Room은 필요한 서비스를 멤버로 가진다. 구현하다 보니 룸의 기본기능 뿐 아니라 플레이어들과 호스트 권위, 게임의 라이프사이클을 관리하는 기능 등 Room의 책임이 너무 많아져 헤더파일만 해도 1000줄이 넘어가서 기능을 확장하기 어려워지는 문제가 생겼었다.

모든 기능을 룸에 구현하는 대신 구성 관계(Has-A)로 책임을 나눴다. 방 상태의 변경 진입점은 Room에 두면서, 각 기능의 계산과 진행은 해당 서비스가 맡는다.

요청 처리기는 방 상태를 소유하지 않는다

패킷 처리기는 요청 종류에 맞는 담당 처리기를 선택한다. 방 요청 처리기는 세션과 요청 형식을 확인하고, 방 목록 관리자에서 대상 Room을 찾는다. 처리기는 방에 수행할 작업을 전달하고, Room과 기능별 서비스가 참가자 여부·준비 상태 등 방 규칙을 적용한다.

입장 요청을 처리할 때 서버는 새 참가자에게 초기 방 상태를 응답한다. 기존 참가자에게는 입장 알림을 보내 명단을 갱신한다. 요청 검증, 방 상태 변경, 응답과 알림 전송을 각각의 책임으로 나눴다.

여러 실행 주체의 작업을 방별로 직렬화한다

IOCP worker 여러 개가 같은 방의 입장·퇴장·준비 요청을 처리할 수 있다. 타이머도 게임 진행 작업을 전달한다. 이 실행 주체들이 방 상태를 동시에 변경하면 참가자 목록과 게임 진행 상태 사이의 순서를 유지하기 어렵다.

서버는 방 상태를 바꾸는 작업을 해당 Room의 작업 큐에 넣는다. 같은 방에서는 한 실행자가 작업을 차례로 처리한다. 서로 다른 방은 각자의 큐에서 독립적으로 작업을 처리한다.

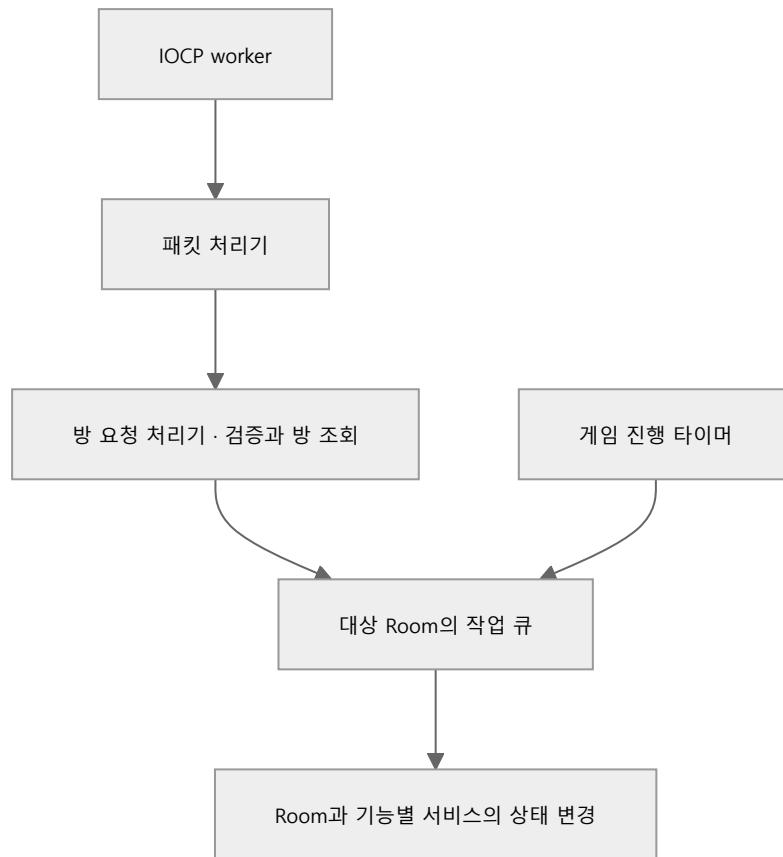
여러 요청을 큐에 모아 처리한다

발표 자료의 JobQueue 장표는 여러 클라이언트에서 전달한 작업이 두 개의 큐에 모이고, 서버가 각 큐의 작업을 처리하는 흐름을 보여준다. 서버 구현에서는 Room마다 작업 큐를 두어 같은 방의 상태 변경 순서를 유지했다.



같은 방으로 들어온 작업은 해당 큐에 쌓인 순서대로 처리한다. 서버는 방마다 작업의 실행 순서를 관리한다.

수신부터 상태 변경까지



작업 큐에 전용 스레드를 만들지는 않았다. 작업을 넣은 호출자는 실행자가 없으면 큐 처리를 시작한다. 이미 실행자가 있으면 새 작업은 큐에서 차례를 기다린다. 따라서 Room 작업의 실행 스레드는 작업을 전달한 worker나 타이머 호출 경로에 따라 달라질 수 있다.

큐는 작업을 넣고 꺼낼 때 lock으로 큐와 실행 상태를 보호한다. 실행자는 작업을 꺼낸 뒤 lock을 놓고 해당 작업을 수행한다. 큐의 실행 상태를 유지하므로 다른 호출자가 같은 방의 큐를 동시에 실행하지 않는다.

방과 서비스의 수명 연결

방 목록 관리자는 방 객체를 강한 참조로 보관한다. Room이 보유한 게임 진행 서비스는 Room을 약한 참조로 가리킨다. 이벤트를 전달받는 네트워크 어댑터도 Room을 약한 참조로 가리켜 순환 소유를 피한다.

이 구조에서 방 목록 관리자는 방의 생성과 보관을 맡고, Room은 상태 변경 경계를 맡는다. 기능별 서비스는 게임 규칙을 나누고, 작업 큐는 같은 방의 변경 순서를 유지한다. 책임 분리와 작업 직렬화를 함께 구성한 결과다.

게임 진행 결과를 클라이언트 알림으로 연결하는 구조는 [이벤트 전달 구조](#)에서 이어진다.

05 / Refactoring

게임 로직과 네트워크를 분리한 이벤트 전달 구조

게임 진행 서비스가 만든 이벤트를 어댑터가 네트워크 알림에 연결한다. Unity에서는 방 서비스가 수신 알림으로 방 상태를 갱신하고 UI에 변경 이벤트를 전달한다.

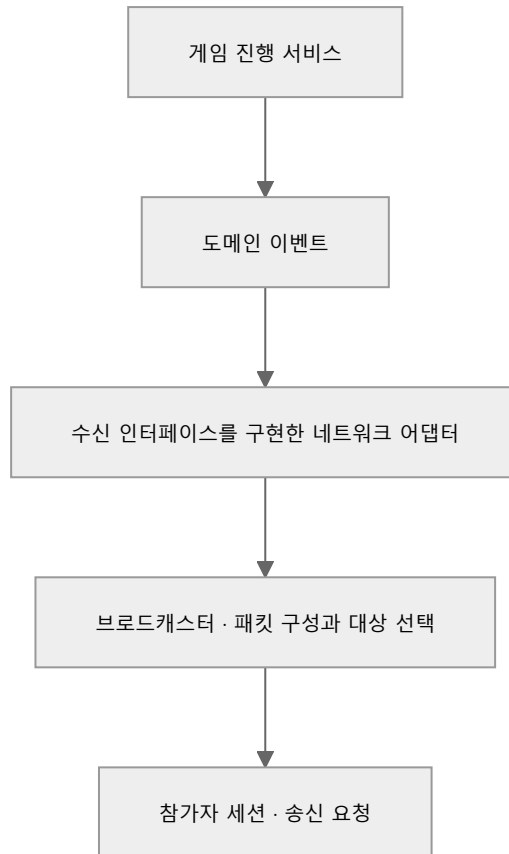
[블로그 원문 ↗](#)

게임 규칙과 전송 책임을 나눈다/ 어댑터 패턴

게임 진행 서비스는 라운드와 페이지를 계산하고, 그 결과를 도메인 이벤트로 만든다. 서비스는 인터페이스 타입으로 어댑터 객체를 가진다. 서비스가 이벤트를 생성해 Publish하면, 어댑터의 `std::visit`가 이벤트 타입에 맞는 람다를 호출하고 브로드캐스터에 연결(호출)한다.

즉, 어댑터 자체가 이벤트 수신자(sink)다. 게임 진행 서비스는 수신 인터페이스 타입으로 어댑터를 보유하고, 이벤트를 발행할 때 그 어댑터를 호출한다. 인터페이스와 구현체를 통해 게임 로직과 네트워크 전송의 경계를 나눈 Port-Adapter 구조다.

구성	역할
게임 진행 서비스	게임 규칙과 상태를 계산하고 결과 이벤트 생성
이벤트 수신 인터페이스	게임 진행 서비스가 외부에 결과를 전달하는 접점(Port)
네트워크 어댑터	수신 인터페이스를 구현하고 이벤트 종류를 전송 동작(람다)에 연결(Adapter)
브로드캐스터	결과를 패킷으로 구성하고 대상 세션을 선택해 송신 요청
참가자 세션	연결된 클라이언트로 패킷 전송



이벤트 객체와 처리 함수를 연결한다

서버는 여러 종류의 이벤트 객체를 `std::variant` 하나로 전달한다. 어댑터는 각 이벤트 타입을 매개변수로 받는 람다 함수를 준비한다. `std::visit`는 전달된 이벤트의 실제 타입에 맞는 람다 함수를 호출한다.

즉, 게임 진행 서비스가 결과 이벤트를 발행하면 어댑터는 그 이벤트에 대응하는 처리 함수를 실행한다. 처리 함수는 이벤트에 담긴 값을 적절한 브로드캐스터에 넘긴다. 이벤트 타입과 처리 함수의 연결은 컴파일할 때 정해진다.(이벤트 타입에 맞는 처리 함수가 오버로딩 규칙에 따라 결정된다)

이 방식으로 이벤트 종류에 따른 처리 분기를 어댑터에 모았다. 게임 진행 서비스는 게임 상태와 결과를 계산하고 게임 규칙을 판단하는 역할을 맡고, 브로드캐스터는 이미 결정된 결과의 전송을 맡는다.

이벤트 전달과 비동기 송신의 실행 순서

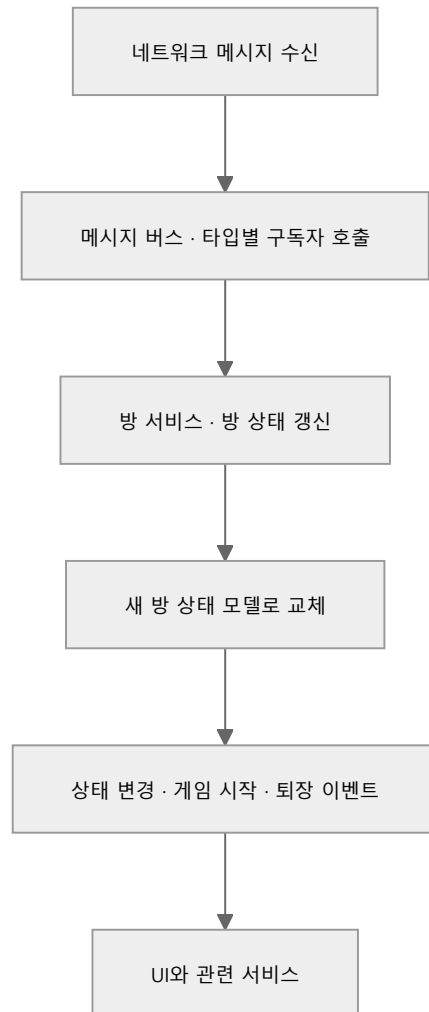
게임 진행 서비스는 Room 작업 큐에서 상태를 변경하고 이벤트를 만든다. 서비스는 자신의 상태 lock을 놓은 뒤 등록된 어댑터를 호출한다. 어댑터와 브로드캐스터 호출도 같은 작업의 실행 흐름에서 이어진다.

브로드캐스터는 방 참가자 목록에서 연결된 세션을 찾아 송신을 요청한다. 이후 Windows가 비동기 I/O를 완료하면 IOCP worker가 완료 통지를 받아 처리한다. 도메인 이벤트의 전달과 소켓 I/O 완료는 이 순서로 연결된다.

Unity의 이벤트 버스 — 발행·구독 패턴

Unity의 네트워크 메시지 버스는 메시지 타입별로 구독한 처리 함수들을 보관한다. 네트워크 수신 경로가 메시지를 발행하면, 버스는 해당 타입의 구독 목록을 찾아 각 처리 함수를 호출한다. 발행자는 메시지를 사용할 UI나 서비스를 직접 참조하지 않는다.

서버 어댑터의 `std::visit`는 한 이벤트에 맞는 처리 분기를 선택한다. Unity의 메시지 버스는 한 메시지를 구독한 여러 처리 함수에 전달한다. 서버와 클라이언트는 각각의 역할에 맞게 이벤트를 연결했다.



방 서비스는 참가자·준비·방장 변경 알림을 구독한다. 서비스는 받은 내용으로 새 방 상태 모델을 만들고 기존 모델을 교체한 뒤 변경 이벤트를 발행한다. UI와 관련 서비스는 방 서비스의 이벤트를 구독해 화면과 동작을 갱신한다.

방 서비스가 네트워크 알림을 화면에서 사용할 상태로 바꾸는 창구를 맡으므로, UI는 서버 패킷을 직접 해석하거나 참가자 목록을 별도로 관리할 필요가 없다.

구독의 수명을 서비스에 맞춘다

메시지 버스는 구독을 해제할 수 있는 객체를 반환한다. 방 서비스는 이 객체들을 `CompositeSubscription`에 모으고, 서비스가 정리될 때 구독을 한꺼번에 해제한다. 방을 나갈 때는 보관 중인 방 상태를 비우고 퇴장 이벤트를 발행한다.

게임 로직과 네트워크를 분리한 이벤트 전달 구조 · 계속

서버는 도메인 결과를 어댑터를 통해 네트워크에 연결한다. 클라이언트는 네트워크 알림을 방 상태로 바꾼 뒤 UI에 이벤트로 전달한다. 양쪽 모두 계산·전송·화면 갱신의 책임을 나눠 구성했다.

06 / Synchronization

HostTick과 InputTick으로 실시간 입력을 어떻게 맞췄는가

지금 만든 입력을 미래의 호스트 Tick에 예약했다. 호스트 시간 추정, RTT 보정, 확정 신호 기준 제한이 각각 무엇을 막는지 정리했다.

[블로그 원문 ↗](#)

문제 Guest가 현재 Tick의 입력을 보내도 호스트에 도착할 때는 그 Tick이 끝날 수 있었다.

원인 서로 다른 실행 시점과 전송 지연을 입력의 대상 시간에 반영해야 했다.

해결 추정 HostTick에 RTT 절반의 Tick 수와 lead를 더해 미래 입력을 예약했다.

결과 생성·전송·소비·확정 신호를 InputTick 하나로 연결해 deadline을 추적할 수 있게 됐다.

역할 입력 시간축과 이력 관리, 호스트 Resolve 연결 및 경계 계측을 담당했다.

용어 정리

용어	뜻
Tick	호스트가 물리 계산을 한 번 하는 단위(초당 60번, 한 번에 약 16.67ms)
Guest / Host	Guest는 일반 참가자, Host는 물리 판정을 맡은 방장 PC
RTT	패킷이 갔다가 돌아오는 데 걸리는 시간
lead	네트워크 지연 외에 추가로 주는 여유 Tick 수
Resolve	호스트가 그 Tick에 쓸 입력을 하나로 확정하는 것
fallback	그 Tick에 받은 입력이 없어서 이전 값이나 중립 값을 대신 쓰는 처리
stale	이미 판정이 끝난 Tick을 대상으로 뒤늦게 온 입력
deadline	그 Tick이 입력을 받아줄 수 있는 마감 시점
batch	여러 Tick의 입력을 한 패킷에 묶은 것
backfill	Guest가 놓친 Tick의 입력을 뒤늦게 채워 넣는 것
Prediction	서버 응답을 기다리지 않고 내 화면에서 먼저 움직여 보여주는 것
Snapshot	호스트가 확정한 전체 상태를 모두에게 보내는 묶음

현재 만든 입력은 미래에 도착한다

멀티플레이 입력을 처음 연결할 때는 버튼과 이동 방향을 보내는 것보다, 그 입력을 호스트가 언제 적용해야 하는지가 중요했다. Guest가 보는 현재와 호스트가 실행 중인 현재는 완전히 같지 않다. Guest에서 읽은 입력이 서버 중계를 거쳐 도착하는 동안 호스트의 물리 시뮬레이션은 계속 진행된다.

HostTick과 InputTick으로 실시간 입력을 어떻게 맞췄는가 · 계속

60Hz에서는 한 Tick이 약 16.67ms다. 입력이 도착하기까지 한두 Tick만 지나도, 보낸 쪽에서 현재라고 붙인 번호를 호스트는 이미 처리했을 수 있다. 늦게 도착한 command를 임의의 다음 Tick에 적용하면 입력 생성 순서와 물리 결과를 대조하기 어려워진다. 재전송한 입력을 새 입력으로 오인할 위험도 생긴다.

그래서 HostTick을 물리 시뮬레이션의 기준 시간으로 두고, InputTick을 “이 입력이 실행될 예약 Tick 번호”로 따로 붙였다. Guest가 화면에 표시 중인 프레임 번호를 그대로 보내는 대신, 이 입력이 호스트의 어느 물리 Tick에서 쓰일지를 먼저 정해서 보냈다.

HostTick과 InputTick이 맡은 역할

HostTick은 “호스트가 지금 몇 번째 물리 계산을 하고 있는가”를 나타내는 숫자다. InputTick은 “이 입력을 몇 번째 계산에서 써 달라”고 붙이는 `span class="notice-yellow">목표 예약 번호</span>다. 둘 다 같은 기준 시간(호스트의 Tick)을 쓰지만, 입력을 막 만든 순간에는 보통 InputTick이 조금 더 미래를 가리킨다.`

값	의미	쓰이는 곳
HostTick	호스트가 지금 처리 중인 Tick, 기준 물리 시간	물리 계산, Snapshot 기준
EstimatedHostTick	Guest가 짐작한 지금의 호스트 Tick	새 입력에 붙일 번호를 정할 때
InputTick	이 입력을 적용하고 싶은 목표 Tick	입력 버퍼, 중복·stale 판별
LastResolvedInputTick	호스트가 마지막으로 처리를 끝낸 Tick	이력 정리, 전진 범위 제한

이 구분은 재전송에서도 필요했다. 같은 InputTick을 다시 받으면 새 물리 입력을 한 번 더 실행하는 대신 중복으로 판별할 수 있다. 반대로 버퍼에 처음 들어왔더라도 이미 Resolve한 Tick보다 뒤쳐져 있으면 stale로 처리한다. 그래서 패킷이 도착한 순서와 이 입력이 적용될 Tick 번호를 서로 다른 것으로 다뤘다.

또한 Tick만으로 모든 세션을 식별하지 않았다. 방이 재사용되거나 호스트가 변경되면 같은 번호가 다른 실행을 가리킬 수 있다. RoomIdentifier와 WorldEpoch, AuthorityEpoch를 함께 확인해 이전 세계나 이전 권한의 입력이 현재 시뮬레이션에 섞이지 않게 했다.

Guest에서 호스트 시간을 추정했다

EstimatedHostTick

Guest는 수신한 호스트 시간 정보와 자신의 경과 시간을 이용해 현재 HostTick을 추정한다. 개념적으로 기준점의 HostTick에 로컬에서 흐른 시간을 시뮬레이션 주기로 환산해 더한다. 각 PC의 벽시계를 일치시켜 놓고 계산하는 구조에 의존하지 않았다.

TEXT

```
EstimatedHostTick ≈ anchorHostTick + floor(localElapsedSeconds × simulationHz)
```

이 값은 호스트에서 직접 읽은 현재값과 차이가 날 수 있다. 기준점을 받은 시점의 지연과 로컬 실행 간격이 포함되기 때문이다. 새 스냅샷이나 시간 정보는 이 기준을 갱신하는 데 사용하고, 과거에 만들어 이미 보낸 command의 InputTick은 해당 입력의 식별자로 유지했다.

HostTick과 InputTick으로 실시간 입력을 어떻게 맞췄는가 · 계속

시간 추정과 입력 전송은 서로 다른 문제다. 시계 추정이 잘 맞아도 생성 직후 command가 큐에 오래 머물면 deadline을 넘길 수 있다. 반대로 전송이 빠르더라도 기준 Tick을 너무 과거로 잡으면 도착 즉시 stale이 된다. 두 경로를 각각 예측할 수 있어야 원인을 좁힐 수 있었다.

RTT와 기본 lead로 도착할 시간을 확보했다

networkTicks Offset

새 입력의 예약값은 추정 HostTick에 편도 지연 추정과 기본 lead를 더하는 형태다. 실제 계산에는 왕복 시간의 절반을 사용하고, Tick 수를 올림해 정수 시간축으로 옮겼다.

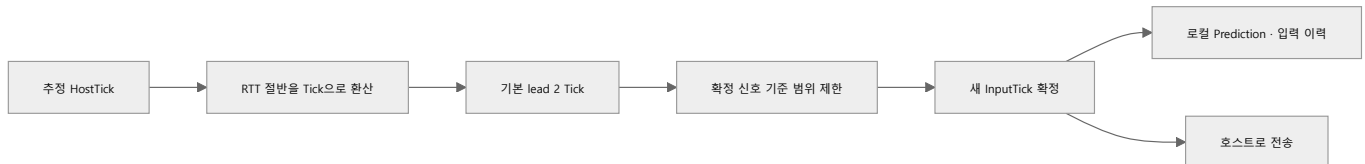
TEXT

```
networkTicks = ceil(RoundTripSeconds × 0.5 × SimulationHz)
NextInputTick = EstimatedHostTick + networkTicks + inputLeadTicks
```

RTT/2는 양방향 지연이 비슷하다는 가정 아래의 추정이다. 송신·수신 경로가 비대칭이거나 프레임 대기가 포함되면 실제 편도 지연과 차이가 난다. 이 값을 편도 지연을 직접 측정한 결과처럼 설명하지 않았다.

LeadTicks Offset

기본 lead는 계산한 지연 외에 실행 순서와 짧은 변동을 받아 줄 여유다. 코드의 `HostAuthoritySession.DefaultInputLeadTicks`는 2다. 60Hz 기준 약 33.3ms에 해당한다. 이 값이 모든 장치와 RTT에서 최적임을 입증한 실험은 없으며, 실제 deadline 계측을 보면서 조정해야 하는 정책값으로 두었다.



계산 예로 추정 HostTick을 100, 편도 추정을 1 Tick이라고 가정하면 lead 2를 더한 후보는 103이다. 호스트가 103을 결정하기 전에 입력이 버퍼에 들어오도록 시간을 배정한다.

미래로 보내는 범위도 제한했다

RTT가 커질 때마다 대상 Tick을 계속 미래로 밀면, 호스트의 처리 진도와 Guest의 입력 이력이 멀어질 수 있다. 아직 확정되지 않은 입력이 길게 쌓이고, 시간 추정의 일시적인 변동이 큰 Tick 점프로 이어질 수 있다. 그래서 미래 예약값에는 별도의 전진 제한을 적용했다.

현재 구현의 `PlayerInputStream.MaximumResolvedTickLead`는 12다. `ConstrainGuestTick`은 마지막으로 받은 확정 신호를 기준으로, 아직 그 신호를 못 받았다면 시작 HostTick을 기준으로 범위를 정한다. 허용 범위는 기준값 다음 Tick부터 기준값 +12까지다.

설정	현재 값	기준
DefaultInputLeadTicks	2	추정 HostTick + 편도 추정 뒤에 더하는 여유
MaximumResolvedTickLead	12	마지막 확정 신호 또는 시작 HostTick으로부터의 최대 전진

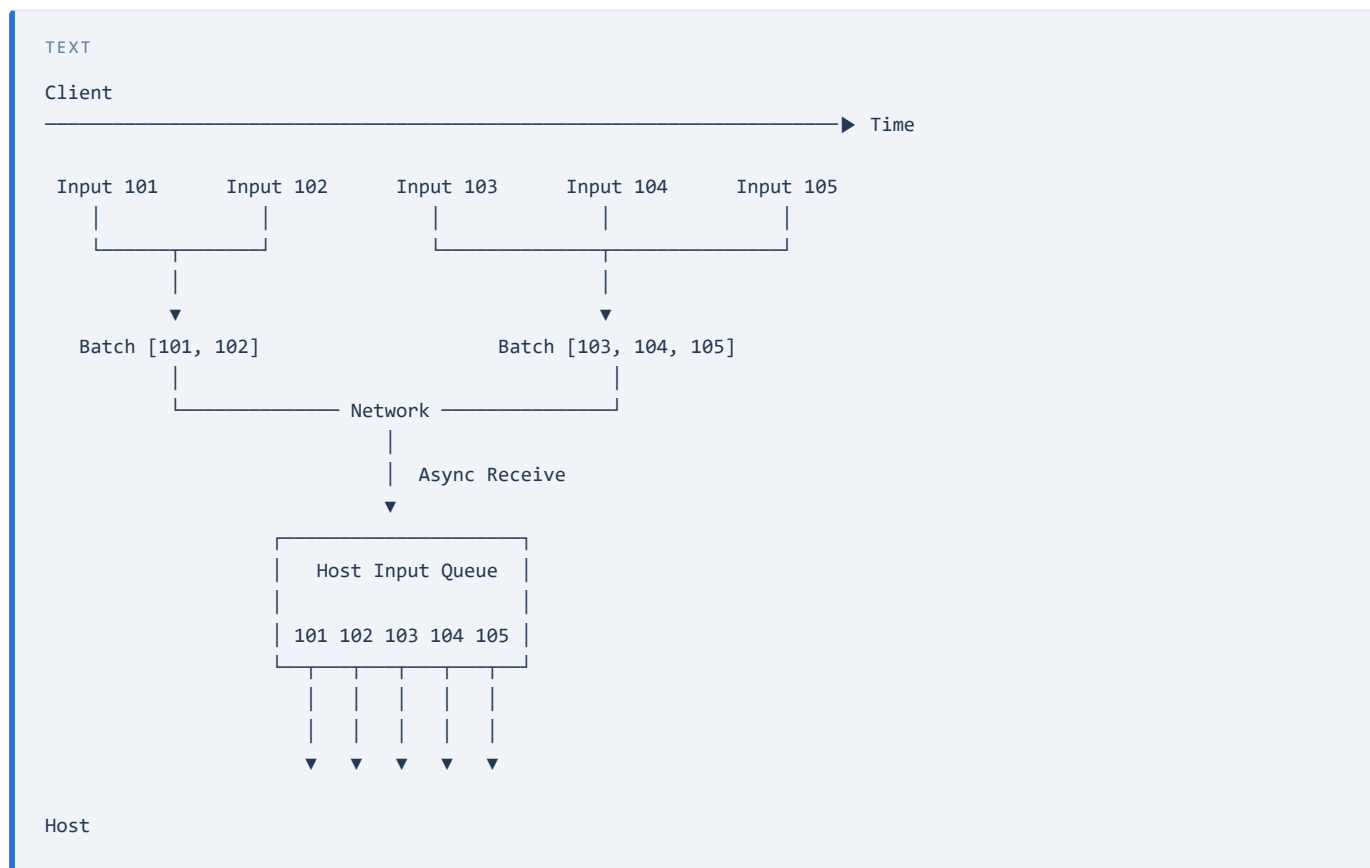
HostTick과 InputTick으로 실시간 입력을 어떻게 맞췄는가 · 계속

이 두 값은 서로 다른 것을 제한한다. "기본 lead의 최대치가 12"라는 뜻으로 읽으면 계산이 달라진다. 12 Tick은 60Hz 기준 200ms지만, 확정 신호를 늦게 받으면 기준점도 과거에 머무른다. 따라서 지금 호스트 시각 기준으로 항상 200ms까지 미래 입력을 허용한다고 볼 수도 없다.

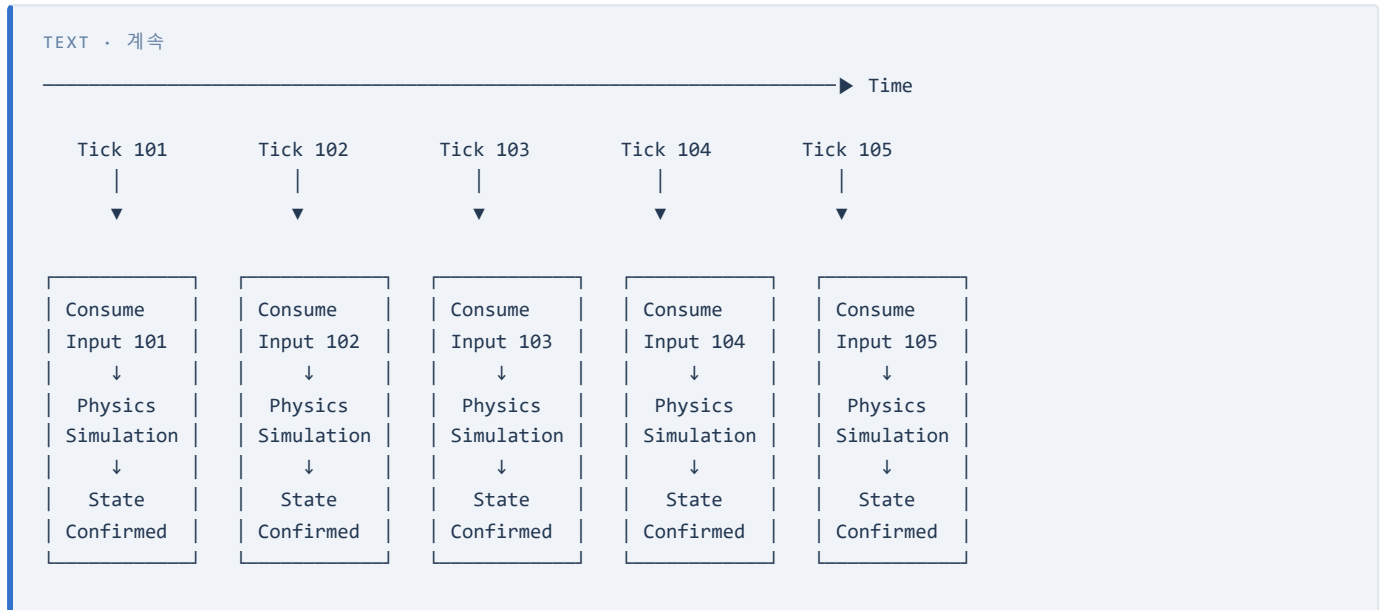
제한을 두면 이력이 너무 앞서 쌓이는 건 막을 수 있지만, RTT가 높을 때 원하는 만큼 미래로 보낼 수 없다는 비용이 생긴다. 숫자를 무조건 크게 만든다고 deadline을 놓치는 문제가 다 풀리지도 않아서, 확정 신호가 얼마나 잘 나아가는지와 후보 Tick이 얼마나 자주 막히는지도 함께 봐야 한다.

60Hz 입력과 30Hz 송신을 나눴다

시뮬레이션은 60Hz로 진행하고 일반 입력 전송은 30Hz batch로 묶었다. 한 패킷이 반드시 한 Tick을 뜻하지 않는다. batch에는 새 입력과 최근 미확정 입력이 함께 들어갈 수 있고, 호스트는 각 command의 InputTick을 기준으로 삽입 여부를 판단한다.



HostTick과 InputTick으로 실시간 입력을 어떻게 맞췄는가 · 계속



두 batch로 나눠 보낸 101~105가 Host에서는 각 Tick 한 개씩 순서대로 소비된다. 그림의 “Host Input Queue”는 별도 클래스 이름이 아니라 이 글에서 계속 쓴 네트워크 수신 큐([InputBatches](#))를 가리킨다. Pump가 이 큐에서 batch를 꺼내 Entity별 [HostInputBuffer](#)로 옮기면, 그다음에야 각 Tick의 [ResolveTick](#)이 해당 InputTick을 찾아 소비한다.

Guest의 렌더 Update가 잠시 멈추면 입력 생성이 권위 시간축을 따라가지 못할 수 있다. 이때 누락된 Tick을 채우는 backfill은 입력 공급을 이어 주는 역할을 한다. 하지만 그 입력이 이미 끝난 호스트 Tick을 대상으로 하면, 많이 생성해 보내도 실제 소비율은 올라가지 않는다. 공급량과 도착 deadline을 분리해 보아야 하는 이유다.

Jump처럼 한 번 발생한 edge에는 정기 batch까지 기다리는 시간이 특히 중요했다. 새 Tick에 edge를 부착하는 순간 최초 송신을 요청하는 경로를 추가했다. 시간 예약 정책을 바꾸는 일과 첫 전송 대기를 줄이는 일을 분리해, 어떤 변경이 어느 경계의 지연을 줄이는지 확인했다.

호스트가 보내는 “여기까지 끝났다”는 신호

LastResolvedInputTick

호스트는 각 Tick에서 실제 입력이 있으면 그걸 쓰고, 없으면 정해진 fallback으로 시뮬레이션을 진행한다. 그 결과인 [LastResolvedInputTick](#), 즉 “여기까지는 확정했다”는 신호를 받은 Guest는 끝난 입력 이력을 정리하고, 아직 안 끝난 입력만 다시 예측 대상으로 남긴다. 이 신호가 입력 이력을 어디까지 지워도 되는지 정하는 기준이 된다.

이때 “끝났다”와 “실제로 썼다”를 혼동하면 안 된다. fallback으로 그냥 넘어간 Tick도 이 신호는 앞으로 나아간다. Guest가 입력을 열심히 만들어서 보낸 양이 많아 보여도, 호스트가 그 입력을 실제로 썼는지는 Resolve 결과를 따로 봐야 한다. [입력 deadline](#) 글에서는 이 기준으로 소비율을 다시 계산했다.

실행 로그에서도 평균은 높는데 가끔 크게 멈추는 구간이 함께 나타났다. 평균만 보면 짧은 정지나 특정 Jump 하나의 실패가 문힐 수 있었다. Tick마다 번호를 유지해 둔 덕분에, 문제가 된 그 한 번이 어느 지점에서 끝났는지 따로 찾아볼 수 있었다.

HostTick과 InputTick으로 실시간 입력을 어떻게 맞췄는가 · 계속

검증 범위와 다음 조정

요약

이번 설계는 입력을 호스트 시간축에 예약하고, 전송·중복·소비·보정을 같은 Tick으로 연결하는 기반을 만들었다. 후속 4인 실행에서 확인한 소비율은 Pump 순서와 송신 정책까지 포함한 파이프라인의 결과다.

동적 InputTick 예약

남은 과제는 RTT 평균뿐 아니라 jitter, 확정 신호가 늦게 오는 정도, 최근 deadline miss까지 함께 반영하는 동적 여유 정책이다. 지금은 기본값과 전진 제한을 두고 운영하며, 환경별로 적정 lead를 자동 탐색하는 완결된 정책까지 확장하지 않았다. 순간적인 RTT 상승에 과도하게 반응하지 않으면서도 늦는 입력을 줄이려면 증가·감소 속도와 관측 구간을 함께 설계해야 한다.

다음 실험 측정 대상

다음 실험에서는 고정 lead별 fallback, 실제 소비율, 미확정 이력 길이와 표시 지연을 같은 조건에서 비교하려 한다. 여유를 더 주면 deadline을 지킬 가능성은 높아지지만 호스트 판정까지의 간격도 길어진다. 이 비용을 수치로 같이 남겨야 플레이 반응성과 안정성 사이에서 설정을 결정할 수 있다.

07 / Synchronization

호스트 권위 멀티플레이에서 Prediction·Reconciliation·Interpolation을 분리한 이유

즉시 움직여야 하는 내 캐릭터와 과거 상태를 재생하는 상대 캐릭터. 물리의 기준점을 지키면서 화면을 부드럽게 연결했다.

[블로그 원문 ↗](#)

문제 예측 위치와 호스트 확정 위치가 달라지고, 물리 갱신 사이에 캐릭터가 끊겨 보였다.

원인 입력, 권위 시뮬레이션, 스냅샷 수신, 렌더링의 시간축이 달랐다.

해결 물리 상태 복원·재예측과 VisualRoot의 화면 보정을 분리했다.

결과 호스트 화면의 Guest 위치 보간 실험에서 표시 속도 변동이 줄었다.

역할 클라이언트 동기화 흐름 구현, 표시 위치 계측 및 보간 전후 비교를 담당했다.

용어 정리

용어	뜻
Guest / Host	Guest는 일반 참가자, Host는 물리 판정을 맡은 방장 PC
Tick / HostTick	호스트가 물리 계산을 한 번 하는 단위(초당 60번)와 그 번호
Prediction	서버 응답을 기다리지 않고 내 화면에서 먼저 움직여 보여주는 것
Reconciliation	내가 예측한 위치를 호스트가 확정된 값에 맞춰 다시 바로잡는 과정
Interpolation(보간)	두 위치 사이를 부드럽게 이어서 보여주는 것
Snapshot	호스트가 확정된 전체 상태를 모두에게 보내는 묶음
PhysicsRoot	충돌·이동 판정에 쓰는 캐릭터의 실제 물리 위치
VisualRoot	화면에 그리기만 하는 캐릭터 모델의 위치
러버밴딩	캐릭터가 갑자기 뒤로 당겨지듯 튀어 보이는 현상
fallback	그 Tick에 받은 입력이 없어서 이전 값을 대신 쓰는 처리
LastResolvedInputTick	호스트가 “여기까지는 확정했다”고 알려주는 값

같은 캐릭터에도 여러 시점의 위치가 있다

로컬 캐릭터 이동 판정에서의 문제

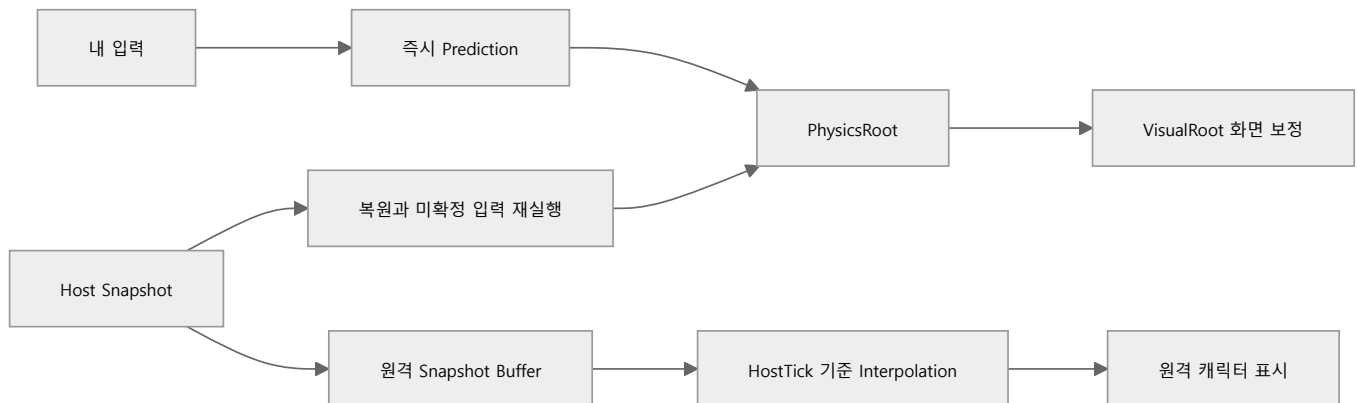
SSketch의 물리 판정은 Unity 호스트가 담당한다. Guest가 이동 키를 누른 시점과 그 입력을 호스트가 처리하는 시점 사이에는 왕복 통신과 실행 순서에 따른 간격이 생긴다. 호스트의 응답을 받은 뒤 움직이면 이 간격이 조작 지연으로 드러난다. Guest에서는 입력을 생성한 즉시 자신의 캐릭터를 예측 이동시켰다.

호스트 권위 멀티플레이어에서 Prediction-Reconciliation-Interpolation을 분리한 이유 · 계속

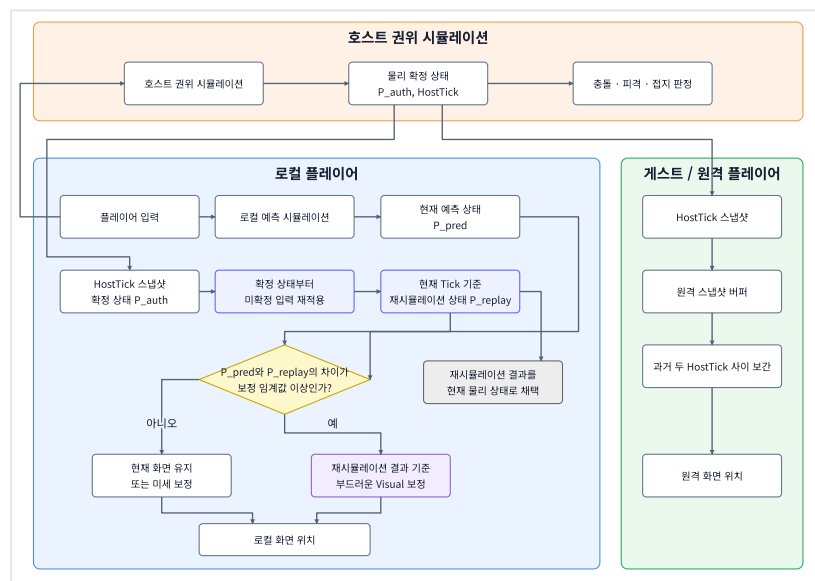
이렇게 미리 움직인 위치는 나중에 호스트가 확정한 위치와 달라질 수 있다. 입력이 도착한 시점이 다르거나, 부딪힌 상대의 상태가 달랐거나, 호스트가 그 Tick에서 fallback을 썼을 수도 있다. 그러다 스냅샷을 받으면 그 확정 상태에 맞춰 예측을 바로잡아야 하는데, 화면까지 그 자리에서 바로 옮겨버리면 캐릭터가 갑자기 뒤로 튕기는 러버밴딩이 보였다.

상대 캐릭터 이동 판정에서의 문제

상대 캐릭터는 또 다른 조건을 가진다. 내가 앞으로 실행할 입력을 가지고 있지 않으므로, 도착한 스냅샷 사이의 움직임을 재생해야 한다. 로컬 플레이어와 원격 플레이어를 동일하게 부드럽게 움직이는 함수 하나로 묶으면, 어떤 상태가 물리 기준이고 어떤 상태가 표시용인지 흐려졌다.



호스트 확정 상태와 로컬·원격 화면의 연결



물리 기준점과 화면 위치를 나눴다

맨 처음 정한 원칙은 다음 예측은 화면 보간의 영향을 받지 않은 물리 상태에서 출발해야 한다는 것이었다. 클라이언트는 호스트 확정 상태를 복원하고 미확정 입력을 다시 실행해 다음 예측의 기준을 만든다. 물리 위치와 화면 위치를 하나의 Root로 처리하면, 위치를 천천히 보정하든 즉시 보정하든 문제가 생긴다.

호스트 권위 멀티플레이어에서 Prediction-Reconciliation-Interpolation을 분리한 이유 · 계속

1. 물리 Root 자체를 보간하면

- 1-1. 다음 예측이 보간 중인 위치에서 시작한다.

클라이언트가 물리 Root를 보정 목표까지 천천히 옮기는 동안 새 입력이 들어오면, 클라이언트는 아직 보정을 끝내지 못한 위치에서 다음 이동을 계산한다. 호스트 확정 상태에서 미확정 입력을 다시 실행해 얻은 물리 상태에 **화면 보간 오차가 섞여 시뮬레이션으로 되돌아간다.**

- 1-2. 물리 상태를 보정 목표에 즉시 맞추지 못한다.

클라이언트가 화면을 부드럽게 만들려고 물리 Root의 이동을 늦추면, 충돌·접지·공격 판정도 보간 중인 위치를 사용한다. 화면 표시를 위한 지연이 물리 판정의 기준까지 바꾼다.

- 1-3. 새 스냅샷이 올 때마다 보간 목표가 바뀐다.

이전 보정이 끝나기 전에 다음 확정 스냅샷이 도착하면, 클라이언트는 보간 도중의 위치에서 새 목표를 향해 다시 움직인다. 이 과정이 반복되면 물리 Root가 보정 목표를 계속 뒤따라갈 수 있다.

2. 보간 없이 물리 Root를 즉시 고치면

- 1-1. 화면에서도 캐릭터가 순간이동하듯 보인다.

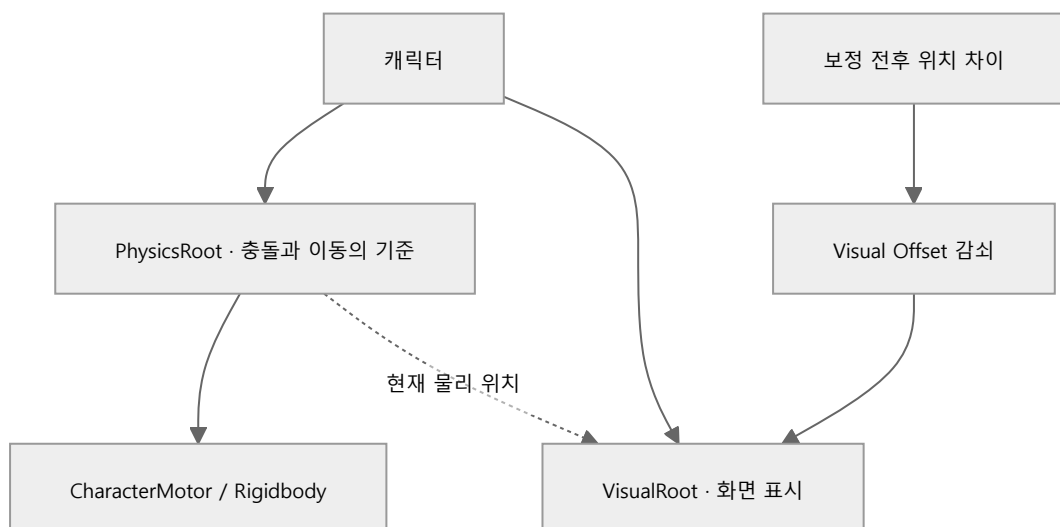
클라이언트가 호스트 확정 상태를 복원하고 미확정 입력을 다시 실행하면 물리 기준은 바로잡을 수 있다. 하지만 화면도 같은 Root를 사용하므로 보정 전후의 위치 차이가 그대로 드러난다.

- 1-2. 보정이 반복되면 화면이 흔들리거나 고무줄처럼 되돌아간다.

클라이언트의 예측 위치와 호스트 기준으로 다시 계산한 위치가 다를 때마다 화면 위치도 즉시 바뀐다. 플레이어에게는 이동 중 캐릭터가 갑자기 앞뒤로 튕는 현상으로 보인다.

물리 보정과 화면 보정을 분리한다

그래서 물리 판정에 쓰는 위치와 화면에 보여주는 위치를 PhysicsRoot와 VisualRoot로 나눴다. 예를 들어 호스트에게서 받은 확정 위치가 10.0m에서 9.9m로 바뀌었다면, PhysicsRoot는 그 즉시 9.9m로 고쳐서 다음 물리 계산에 쓰고, VisualRoot는 화면에서만 0.15초에 걸쳐 천천히 9.9m로 움직인다. 두 위치의 차이는 화면 전용 값으로 남겨두고, 렌더링하면서 서서히 줄여나갔다.



호스트 권위 멀티플레이어에서 Prediction-Reconciliation-Interpolation을 분리한 이유 · 계속

개념적으로 화면 위치는 `physicsPosition + visualOffset`이다. `offset`은 충돌 판정이나 다음 입력의 시작 위치에 섞지 않는다. 한 프레임 동안 예쁘게 보이기 위해 바꾼 좌표가 다음 Tick의 물리 상태로 흘러들어 가지 않도록 경계를 만들었다.

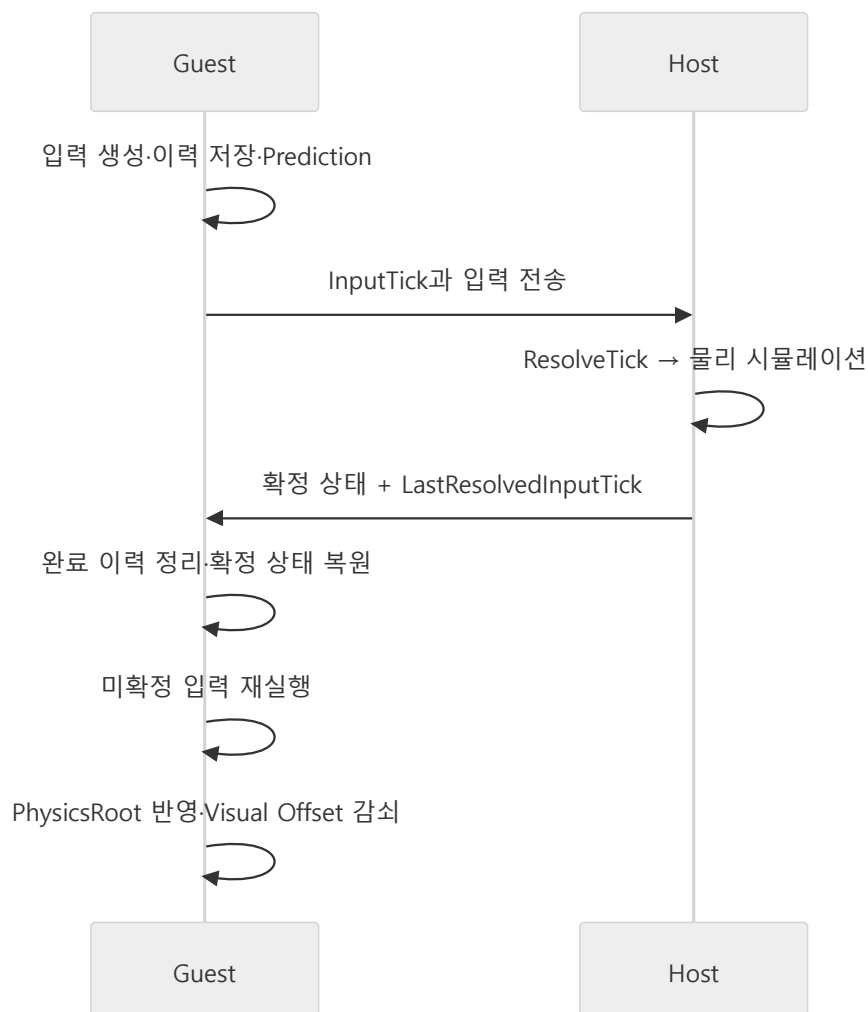
분리한 효과

- 다음 입력은 **화면 보간의 영향을 받지 않은 물리 위치에서 계산한다**.
- 충돌체 위치와 접지 상태는 **물리 보정에 바로 반영된다**.
- 화면에서는 **순간이동이나 떨림이 덜 보인다**.
(입력 fallback이 늘거나 예측 위치 오차가 크게 흔들리면 떨림이 체감될 수 있음)
- 카메라는 **화면에 보이는 캐릭터 위치를 따라가도록 구성할 수 있다**.

모든 오차를 다 오래 보간하는 것도 피해야 했다. 순간이동이나 리스폰처럼 위치가 뚝 끊겨서 바뀌는 상황에서는, 이전 위치와 억지로 이어 붙이면 오히려 이상하게 움직이는 걸로 보인다. 이렇게 위치가 통째로 바뀌는 순간에는 이전에 보간하던 기록을 다 비우고, 새 위치를 바로 기준으로 삼았다.

로컬 플레이어: 확인한 뒤 남은 입력을 다시 실행한다

로컬 Guest는 InputTick과 입력 내용을 이력에 남기고 즉시 Prediction을 수행한다. 스냅샷을 받으면 호스트가 어디까지 처리했는지 확인해 완료된 입력을 정리한다. 이어서 호스트 확정 상태를 복원하고, 아직 확정되지 않은 입력을 순서대로 다시 적용한다. 최종 예측 상태가 새 PhysicsRoot의 기준이 된다.



`LastResolvedInputTick`을 해석할 때는 주의가 필요했다. 호스트가 그 Tick을 끝냈다는 사실과 실제 입력을 소비했다는 사실은 구분해야 한다. fallback으로 처리한 Tick도 시간축은 앞으로 진행한다. 이미 끝난 Tick의 점프를 재전송한다고 과거 판정이 다시 열리지는 않는다. 점프가 예측에서는 실행됐는데 호스트에서는 실행되지 않은 문제는 **단발 입력 동기화**에서 따로 추적했다.

현재 Reconciliation은 호스트가 보낸 내 캐릭터의 확정 상태를 기준으로, 아직 확정되지 않은 내 입력만 다시 적용하는 방식이다. 다른 플레이어나 월드 오브젝트의 과거 상태까지 복원하는 전체 Rollback은 수행하지 않는다. 따라서 다른 플레이어와의 충돌처럼 외부 객체가 내 이동 결과에 영향을 준 상황에서는 과거 물리 상황을 완전히 재현할 수 없으며, 이후 호스트 Snapshot을 통해 오차를 다시 보정한다.

TEXT

화면: 예측 위치로 즉시 이동 ————— 부드럽게 보정
 권위: 아직 미확정 — HostTick T에서 물리 확정
 네트워크: 입력 전송 ——— 확정 스냅샷 수신

1. 게스트는 `LastResolvedInputTick` 이하의 입력을 제거하고, 호스트 확정 상태부터 **미확정 입력을 다시 적용**한다.
2. 게스트는 재시물레이션 결과를 **물리 상태에 즉시 반영**하고, 화면 모델만 짧은 보간으로 보정한다.
3. 순간이동-리스폰이나 큰 위치 오차는 **보간하지 않고 즉시 스냅**한다.

Guest가 보는 다른 플레이어는 HostTick이 붙은 스냅샷을 버퍼에 저장했다. 수신 즉시 최신 위치로 옮기는 대신, 표시할 Tick을 약간 과거에 두고 그 시점을 감싸는 두 스냅샷 사이를 보간했다. 패킷 도착 간격이 조금 흔들려도 렌더링은 연속된 위치를 사용할 수 있다.

물리는 60Hz, 스냅샷 전송은 30Hz로 구성했다. 렌더링이 60Hz일 때 새 스냅샷이 없는 프레임도 있으므로, 수신 횟수와 화면 갱신 횟수를 일치시키려 하면 계단식 이동이 드러난다. 보간은 두 갱신 주기 사이를 연결하는 역할을 한다. 실제 렌더 프레임은 장치와 부하에 따라 달라진다.

TEXT

확정 상태: A ————— B ————— C

보간 없음:

A A A A A B B B B B C
└ 같은 위치 반복 ┐ → 다음 위치로 한 번에 이동

보간 적용:

A B C
└ A→B 사이 ┐ └ B→C 사이 ┐
 중간 위치를 계산해 부드럽게 표시

144Hz 화면처럼 상태 갱신보다 화면 갱신이 빠르면, 같은 위치를 여러 프레임 표시하게 된다. 보간은 A-B, B-C 사이의 중간 위치를 계산해 계단식 움직임을 줄인다. 원격 플레이어의 A·B·C는 **30Hz로 받은 확정 스냅샷**이며, 그림의 점은 그 사이에 표시할 위치를 뜻한다.

이 선택에는 표시 지연이 따른다. 두 상태 사이를 안정적으로 보간하려면 뒤쪽 상태가 도착할 시간을 확보해야 한다. 버퍼가 비는 구간에서는 가진 상태만으로 움직임을 무한히 정확하게 예측할 수 없다. 지연이 커졌을 때의 표시 정책과 큰 위치 오차에서의 snap은 별도로 다뤄야 한다.

호스트 권위 멀티플레이어에서 Prediction-Reconciliation-Interpolation을 분리한 이유 · 계속

렌더 Tick에 맞는 두 상태를 고른다

다른 플레이어의 미래 입력은 알 수 없으므로, 게스트는 자신의 캐릭터처럼 입력을 재실행하지 않는다. 게스트는 추정 HostTick에서 보간 지연을 뺀 **과거의 렌더 Tick**을 선택한다.

TEXT

```
renderTick = estimatedHostTick - interpolationDelayTicks
alpha = (renderTick - from.HostTick) / (to.HostTick - from.HostTick)
화면 위치 = Lerp(from.Position, to.Position, alpha)
```

from과 **to**는 렌더 Tick을 앞뒤로 감싸는 두 확정 스냅샷이다. 두 위치 사이의 보간은 **임의의 미래 이동을 만들어 내지 않는다**.

새 스냅샷이 늦어 표시 시점이 최신 상태를 넘으면, 게스트는 마지막 확정 위치와 속도로 **제한된 외삽**을 수행한다. 외삽 시간에 상한을 두어 패킷 공백 동안 캐릭터가 계속 이동하지 않게 한다. 이는 다른 플레이어의 미래 입력을 추측하는 예측과 구분한다.

호스트 화면에서도 Guest가 끊겨 보였다

호스트는 Guest의 물리를 직접 시뮬레이션하므로 스냅샷 수신 지연 없이 최신 권위 상태를 가지고 있다. 그런데도 화면에서 작은 끊김이 보였다. 60Hz 물리 단계 사이에 렌더 프레임이 끼면 같은 물리 위치를 한 번 더 표시하거나, 다음 위치로 한꺼번에 이동할 수 있었다.

이 경로에는 원격 스냅샷 보간과 별도로, 직전·현재 권위 물리 위치 사이를 렌더 시점에 연결하는 위치 보간을 적용했다. 렌더 경계에서 VisualRoot 위치만 바꾸고 표시가 끝난 뒤 복원해 물리 로직으로 영향이 돌아가지 않게 했다. 실험 대상은 위치 보간이며 회전이나 로컬 Prediction 전체를 한 번에 바꾼 실험으로 확대 해석하지 않았다.

물리 Tick 사이의 화면 위치

TEXT

```
물리 Tick:      A ——— B ——— C
보간 없음:      A A A   B B   C C C
                  같은 위치 유지 → 다음 위치로 한 번에 이동

보간 적용:      A . .   B . .   C
                  A-B, B-C 사이의 중간 위치를 화면에 표시
```

144Hz 화면에서는 60Hz 물리 상태 하나가 여러 렌더 프레임에 걸쳐 표시될 수 있다. 호스트는 **권위 물리 Root를 유지**하고, 카메라가 그리는 순간에만 VisualRoot를 이전·현재 물리 위치 사이로 보간한다. 표시 보간은 설정으로 켜고 끌 수 있다.

렌더 한 번의 동작

이전 물리 위치를 A, 현재 물리 위치를 B라고 하면 렌더 한 번의 동작은 다음과 같다.

호스트 권위 멀티플레이에서 Prediction-Reconciliation-Interpolation을 분리한 이유 · 계속

C#SHARP

```
// 1. 렌더 후 복원할 현재 위치 B를 보관
_authoritativeVisualPosition = _visualRoot.position;

// 2. 마지막 물리 갱신 이후 경과 시간으로 진행률 계산
float alpha = Mathf.Clamp01(
    (float)((Time.timeAsDouble - Time.fixedTimeAsDouble) / fixedDeltaTime));

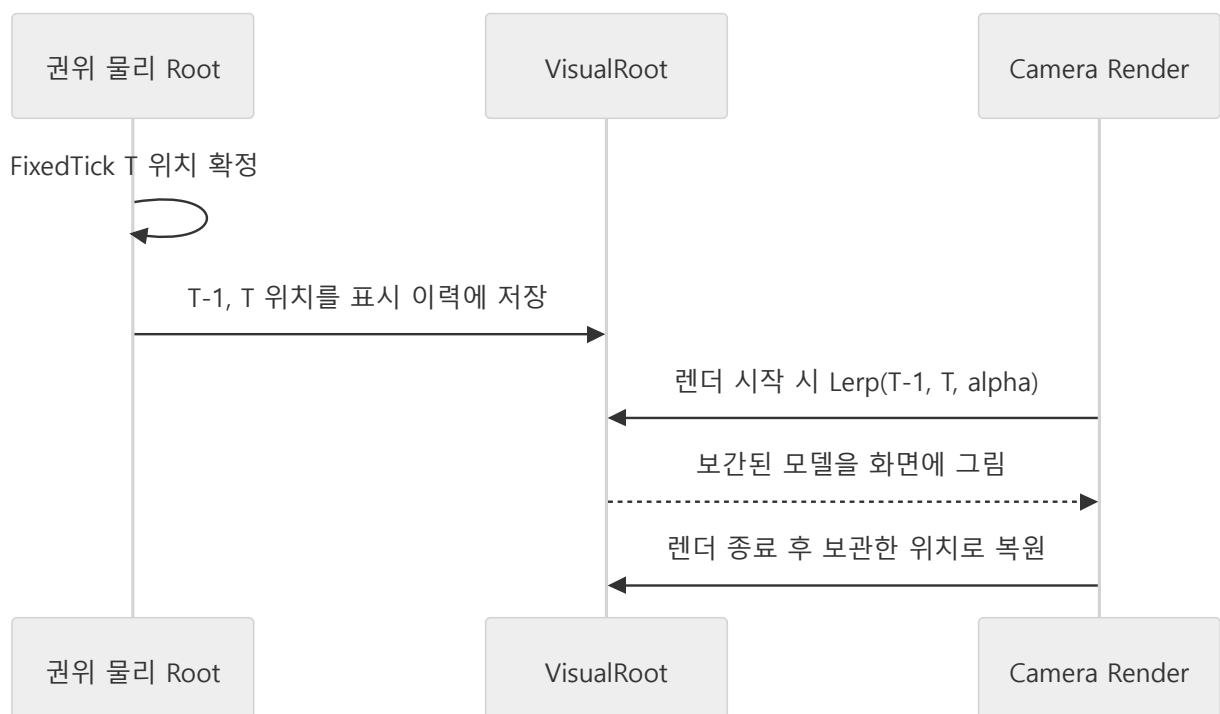
// 3. 화면에 그릴 위치를 임시 적용
_visualRoot.position = Vector3.Lerp(A, B, alpha);

// 4. 카메라가 보간된 모델을 렌더링

// 5. 렌더 종료 후 보관한 위치 B로 복원
_visualRoot.position = _authoritativeVisualPosition;
```

alpha는 렌더 프레임 수가 아니라 시간으로 계산한다. 60Hz의 물리 갱신 간격 약 16.67ms를 기준으로 하면 다음과 같다.

마지막 물리 갱신 후 경과 시간	alpha
2ms	약 0.12
7ms	약 0.42
13ms	약 0.78



공격·충돌·스냅샷 생성은 계속 권위 물리 Root를 사용한다. 화면 모델만 최대 한 물리 Tick 늦게 표시하므로, 화면 보간이 게임 판정에 섞이지 않는다.

호스트 권위 멀티플레이에서 Prediction-Reconciliation-Interpolation을 분리한 이유 · 계속

관찰자에 따라 화면 위치를 선택한다

관찰 상황	화면 위치의 출처	시간 특성
호스트가 자신을 봄	현재 권위 물리 상태	최신 HostTick
호스트가 게스트를 봄	직접 계산한 이전·현재 물리 위치의 표시 전용 보간	최대 약 한 물리 Tick 지연
게스트가 자신을 봄	로컬 예측 + 호스트 확정 상태로 재조정 + 화면 보정	입력에 즉시 반응한 뒤 확정 결과에 맞춤
게스트가 다른 캐릭터를 봄	과거의 두 HostTick 스냅샷 사이 보간	보간 지연만큼 과거

최종 판정은 호스트의 물리 상태를 기준으로 하고, 화면은 관찰자에 따라 예측하거나 과거 상태를 보간한다.

검증: 등속 구간과 프레임 조건을 나눠 비교했다

보간으로 인해 일정한 속도로 이동하는 구간에서 화면 움직임이 얼마나 고르게 이어졌을까??

톡톡 끊겨보이는지/얼마나 부드러운지를 화면 속도 표준편차로 표현했다.

2026년 9월 20일, PC 두 대에서 Host 1명과 Guest 1명으로 비교했다. 동일한 일반 Windows 빌드 23bcf381을 사용했고 Development Build와 Profiler는 사용하지 않았다. 호스트가 보는 Guest를 대상으로, 1초 계측 창에서 권위 평균 속도가 3.95~4.05m/s이고 전체 렌더 프레임이 이동 중인 구간을 골랐다.

로컬 계측 PC · 실험 당시와 동일하게 유지한 환경

항목	구성
CPU	Intel Core Ultra 7 155H · 16코어 / 22논리 프로세서
GPU	NVIDIA GeForce RTX 4050 Laptop GPU
메모리	약 32GB · OS 인식 31.51GiB
OS	Windows 11 Enterprise
Unity	6000.3.22f1
호스트 실행 정책	목표 60 FPS · VSync OFF · 백그라운드 실행 유지

위 사양은 로컬 호스트 PC 기준이다. 목표 FPS는 실행 설정이며 실제 프레임 간격은 별도 계측값으로 비교했다.

호스트 권위 멀티플레이에서 Prediction-Reconciliation-Interpolation을 분리한 이유 · 계속

선택한 등속 이동 구간	보간 OFF	보간 ON
1초 창 수 / 렌더 프레임	94 / 5,664	63 / 3,808
평균 표시 속도	4.030m/s	4.010m/s
표시 속도 표준편차	0.906m/s	0.355m/s
표시 위치가 멈춘 프레임	937개, 1.642%	0개

선택한 구간에서는 표시 속도 표준편차가 약 60.8% 줄었다. 다만 OFF와 ON의 프레임 상태까지 같지는 않았다. 보간 효과와 프레임 간격 차이가 섞일 수 있어, 프레임 간격 p95가 20ms 이하이고 최댓값이 25.5ms 이하인 로그들(1초 구간)만 다시 골라 집계 후 비교했다.

프레임 조건을 추가한 부분집합	OFF	ON
1초 창 수 / 렌더 프레임	33 / 1,998	62 / 3,747
평균 프레임 간격	16.664ms	16.669ms
표시 속도 표준편차	0.404m/s	0.350m/s
표시 위치가 멈춘 프레임 비율	0.250%	0%

이 부분집합에서도 변동은 약 13.4% 감소했다. 두 표를 함께 남겨 전체 선택 구간의 큰 차이를 보간 하나의 효과로 단정하지 않았다. 관측한 결과는 등속 이동의 화면 연속성을 개선했다.

부드러움의 비용과 남은 문제

같은 실험의 창별 추정 표시 지연 p95 중앙값은 10.03ms에서 16.67ms로 늘었다. 이는 위치 표시의 지연 추정치이며 키 입력부터 화면까지의 전체 지연 측정값과 구분했다. 60Hz의 한 Tick은 약 16.67ms이므로, 이전·현재 물리 위치를 연결하는 방식이 갖는 시간 비용을 함께 확인한 셈이다.

보간은 이미 존재하는 상태를 화면에 연결한다. Host hitch로 물리 진행 자체가 멈추거나 입력이 deadline을 넘긴 문제는 해당 경로에서 해결해야 한다. 그래서 입력 소비 문제에는 Pre-Tick Pump를, 예측 오차에는 Reconciliation을, 표시 불연속에는 VisualRoot 보정을 각각 적용했다. 이 경계를 나누자 화면의 끊김을 보았을 때 어느 단계의 데이터를 먼저 확인할지 명확해졌다.

08 / Troubleshooting

UDP 패킷은 도착했는데 왜 입력은 누락됐을까? — Host Tick Deadline 추적

수신 여부와 실제 입력 소비를 나눠 계측했다. ResolveTick 직전에 입력 큐를 반영하고, 남은 deadline miss를 다시 추적했다.

[블로그 원문 ↗](#)

문제 4인 실행에서 이동이 간헐적으로 끊기고 fallback이 21.95~24.14% 발생했다.
원인 수신 큐의 입력이 해당 Tick의 Resolve 전에 InputBuffer에 반영되지 않았다.
해결 권위 Tick 시작 직전에 제한된 입력 큐 처리를 수행하는 Pre-Tick Pump를 넣었다.
결과 같은 PC의 수동 비교에서 fallback 3.09~4.57%, 실제 입력 소비 95.43~96.91%를 관측했다.
역할 입력 단계별 계측, lifecycle 순서 수정, Profiler 분석과 2PC 검증을 담당했다.

용어 정리

용어	뜻
Guest / Host	Guest는 일반 참가자, Host는 물리 판정을 맡은 방장 PC
Tick	호스트가 물리 계산을 한 번 하는 단위(초당 60번)
ResolveTick	호스트가 그 Tick에 쓸 입력을 하나로 확정하는 함수
InputBuffer	호스트가 플레이어별로 받은 입력을 Tick 번호별로 모아두는 곳
fallback	그 Tick에 받은 입력이 없어서 이전 값(Held)이나 중립 값(Neutral)을 대신 쓰는 처리
입력 소비율	호스트가 실제로 받은 입력으로 처리한 Tick의 비율
Pre-Tick Pump	Tick을 확정하기 직전에 이미 도착한 입력을 InputBuffer로 먼저 옮기는 처리
Backfill	입력이 빈 Tick에 이전 입력을 채워 넣는 처리
batch	여러 Tick의 입력을 한 패킷에 묶은 것
stale	이미 판정이 끝난 Tick을 대상으로 뒤늦게 온 입력
RTT	패킷이 갔다가 돌아오는 데 걸리는 시간
2PC	Host와 Guest를 서로 다른 PC 두 대에서 실행한 실험
Update / FixedUpdate	Unity가 화면 프레임마다 / 고정 물리 주기마다 부르는 함수
Physics.Simulate	Unity 물리를 한 단계 진행시키는 함수
Profiler	Unity에서 함수별 실행 시간을 재는 도구

UDP 패킷은 도착했는데 왜 입력은 누락됐을까? — Host Tick Deadline 추적 · 계속

패킷은 보이는데 움직임이 끊겼다

Host 1개와 Guest 3개, 총 4개 프로그램을 같은 PC에서 동시에 실행해 이동을 확인하던 중, 캐릭터가 일정하게 움직이다가 짧게 끊기는 현상을 발견했다.

개발 당시 녹화한 문제 상황. 이동이 일정하게 이어지다가 순간적으로 끊기거나 튜는 걸 볼 수 있다.

처음에는 초당 60번 도는 물리 계산과 별개로 FPS가 떨어지면 `Update` 호출 주기가 불안정해지고, 그 결과 한 프레임 안에서 `FixedUpdate` → `Physics.Simulate`가 몰아서 여러 번 실행되면서 일부 Tick의 입력이 누락된다고 추정했다. 이 추정에 맞춰 입력이 빈 Tick에는 이전 입력값을 잠깐 유지하는 Backfill 방식을 넣어봤지만, 현상은 크게 나아지지 않았다.

호스트의 입력 처리 기록을 다시 보니, 실제 입력 대신 이전 상태를 유지하는 Held fallback과 중립 상태를 쓰는 Neutral fallback이 쌓여 있었다. 초기 비교 구간에서 Guest별 fallback 비율은 약 22~24%였다.

UDP를 쓰고 있어서 처음엔 패킷 유실이나 RTT 증가도 의심했다. 하지만 Guest에서 생성한 입력이 실제로 호스트의 네트워크 수신 버퍼까지 도착해 있는 걸 로그로 확인했다. 패킷이 유실된 게 아니라, 입력이 도착한 시점과 호스트가 그 Tick을 확정하는 시점 사이의 처리 순서가 어긋난 것이었다.

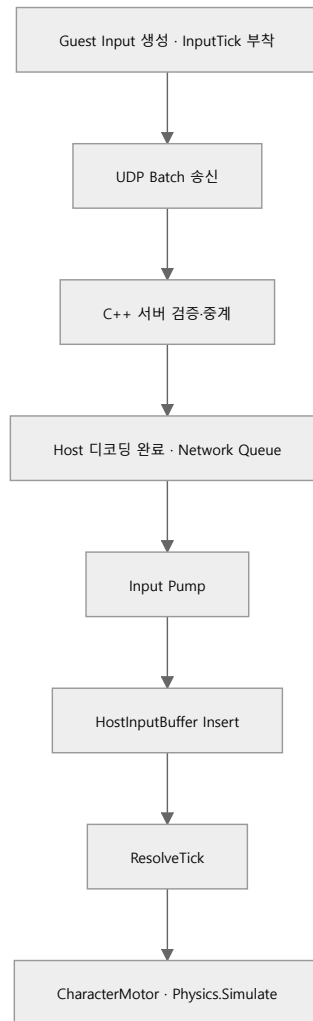
그래서 문제 자체를 다시 정의했다. “왜 이동이 끊기는가”가 아니라 “입력 소비율을 어떻게 끌어올릴 것인가”로 바뀌서 보기로 했다. 네트워크 변수를 최대한 건너내기 위해 로컬 PC의 루프백 주소로 접속한 상태에서도, 클라이언트 로직만으로 입력값이 제대로 소비되는지부터 다시 확인했다.

정확한 원인을 보려고 Guest에서 입력이 만들어진 시점부터 호스트가 그 입력을 실제로 쓰는 시점까지, 단계마다 로그를 남겼다.

입력 생성부터 Resolve까지 경계를 나눴다

입력은 Guest에서 생성된 뒤 곧바로 호스트 물리 함수에 들어가지 않는다. 직렬화와 UDP 전송, 서버 중계, 호스트의 수신·디코딩을 거친다. 이후 네트워크 큐에 보관된 데이터를 메인 실행 흐름에서 꺼내 HostInputBuffer에 삽입해야 ResolveTick이 사용할 수 있다.

UDP 패킷은 도착했는데 왜 입력은 누락됐을까? — Host Tick Deadline 추적 · 계속



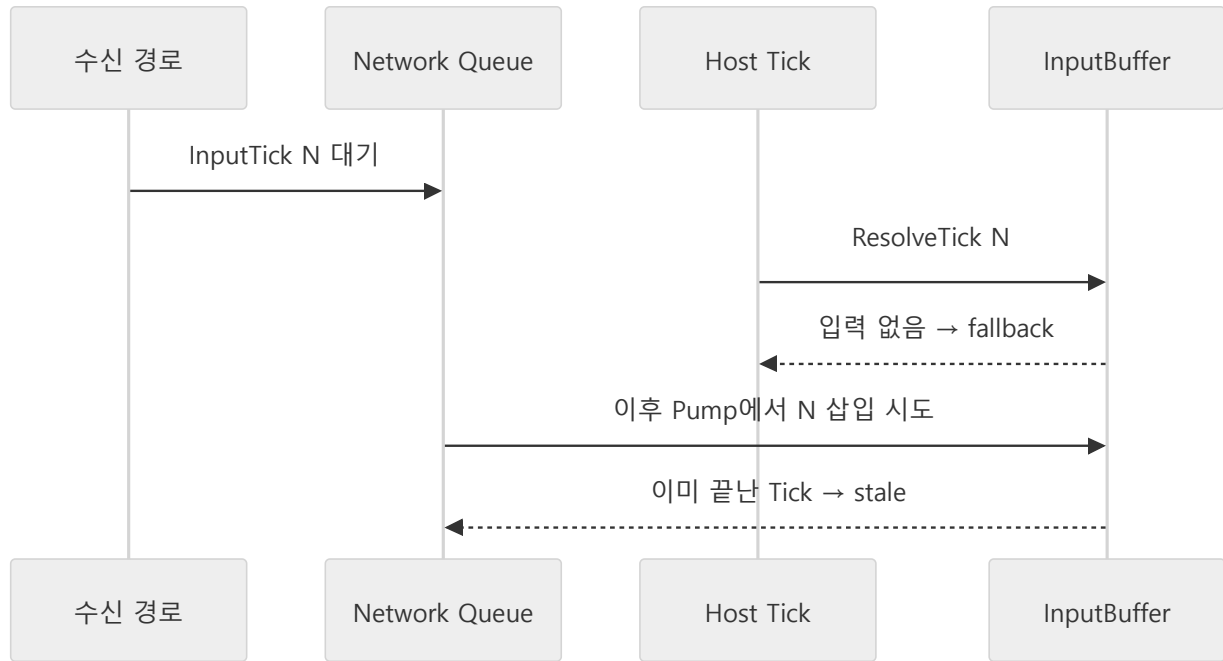
각 단계에는 입력 Tick과 Entity를 함께 남겼다. 방이나 호스트가 바뀌는 경우를 섞지 않도록 Room과 World/Authority epoch도 대조했다. 중복 batch의 행 수를 그대로 더하면 하나의 입력이 여러 번 도착한 것이 여러 개의 새 입력처럼 보이므로, 생성·최초 삽입·최종 결정을 나눴다.

수신 시점의 의미도 고정했다. 후속 2PC 계측에서 사용한 수신 경계는 Unity 호스트에서 디코딩을 마친 뒤 메인 스레드 큐에 넣기 직전이다. NIC나 OS 소켓에 처음 도착한 시점까지 직접 측정한 값으로 확장하지 않았다. 이 경계 이후의 큐 대기과 그 이전의 전송 경로를 구분할 수 있었다.

수신 큐에 있어도 현재 Tick은 fallback이 될 수 있었다

기존 실행 흐름에서는 입력 수신 큐 처리와 권위 Tick의 Resolve 순서가 어긋날 수 있었다. 네트워크 경로에서는 이미 입력을 큐에 넣었지만, 해당 입력을 InputBuffer에 삽입하기 전에 ResolveTick이 호출되는 경우다. Resolve는 버퍼에 있는 상태만 확인하므로 목표 Tick의 입력이 없다고 판단하고 fallback을 선택했다.

UDP 패킷은 도착했는데 왜 입력은 누락됐을까? — Host Tick Deadline 추적 · 계속



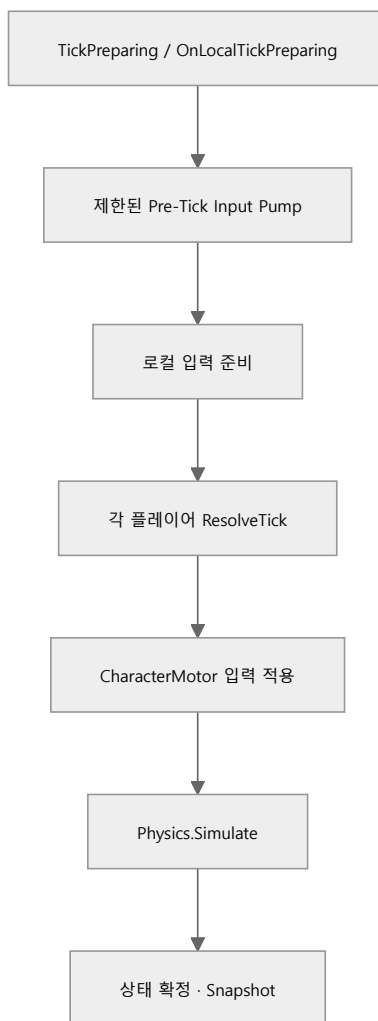
이 순서에서는 네트워크 전송이 빨라져도 메인 실행 흐름에서 반영이 늦으면 같은 문제가 남는다. Guest에서 빠진 Tick을 backfill하거나 재전송 횟수를 늘리는 것만으로도 해결되지 않는다. 입력을 충분히 생성하는 문제와, 생성한 입력을 호스트가 제때 사용하는 문제는 별개의 경계였다.

deadline을 지난 입력을 과거 Tick에 뒤늦게 적용하는 방법도 쉽게 선택할 수 없었다. 호스트는 이미 그 Tick의 물리와 충돌을 진행했고 결과를 다른 Guest에게 전송했을 수 있다. 현재 구조에서는 끝난 Tick을 다시 여는 대신, 진행 중인 Tick이 사용할 수 있는 입력을 먼저 반영하도록 순서를 바꿨다.

Pre-Tick Pump를 권위 Tick 앞에 배치했다

권위 시뮬레이션의 Tick 준비 단계에서 입력 큐를 제한적으로 처리한 뒤, 로컬 입력 준비와 ResolveTick을 진행하도록 연결했다. 중요한 위치는 물리 단계 직전이라는 막연한 표현보다 **해당 Tick의 입력을 결정하기 전**이다. 같은 프레임 안에 들어오더라도 Resolve 뒤에 처리하면 늦다.

UDP 패킷은 도착했는데 왜 입력은 누락됐을까? — Host Tick Deadline 추적 · 계속



큐 처리는 무제한 drain으로 만들지 않았다. 한 Tick을 준비하다가 네트워크 작업에 시간을 계속 사용하면 물리 일정 자체가 밀릴 수 있기 때문이다. 처리 예산과 남은 입력을 관측해, 큐를 비우는 작업이 시뮬레이션 전체를 독점하지 않는지 확인했다. 후속 별도 실행에서는 20,242개 batch와 73,232개 command를 처리하는 동안 budget exhausted와 처리 후 최대 잔여 큐가 모두 0으로 기록됐다.

이 실행에서는 설정한 처리 예산 안에서 큐를 비울 수 있었다. 이전 Profiler 기록과는 입력 수와 실행 시점이 달라 처리 시간 비교를 별도로 남겼다.

또한 Pump 시점까지 큐에 들어온 입력만 현재 결정에 참여할 수 있다. 목표 deadline 이후에 호스트에 도착한 입력은 이 순서 변경으로 복구되지 않는다. 개선 후에도 fallback이 남는 이유를 확인할 때 이 한계를 기준으로 다음 계측을 설계했다.

먼저 지표의 분모를 정리했다

기존 기록에는 입력 생성률, 패킷 수신률, 실제 소비율이 함께 등장했다. 숫자가 비슷해 보여도 의미가 달랐다. 이 글의 소비율은 호스트의 최종 Resolve 결과를 기준으로 계산했다.

TEXT

```

resolvedTicks = receivedResolved + heldFallback + neutralFallback
실제 입력 소비율 = receivedResolved / resolvedTicks
fallback 비율 = (heldFallback + neutralFallback) / resolvedTicks
  
```

UDP 패킷은 도착했는데 왜 입력은 누락됐을까? — Host Tick Deadline 추적 · 계속

반면 Guest 쪽의 “확정 구간 공급률”은 `generatedInAckCohort / expectedAuthorityTicks`로 계산한다. 호스트의 확정 신호가 나아간 만큼, Guest가 그동안 입력을 얼마나 만들었는지를 본다. 생성한 입력이 `deadline`을 지켜 도착했는지까지 보여주지는 않는다. 후속 실험의 공급률 99.49%를 실제 소비율이라고 적으면 개선 결과를 과장하게 된다.

중복 입력 비율도 별도로 봤다. 최근 입력을 묶어 재전송하므로 중복 수신은 설계상 발생한다. Duplicate, stale, fallback의 개수는 서로 다른 사건과 분모를 가지며 단순히 합쳐 패킷 손실률로 계산할 수 없다. 어떤 경계에서 무엇을 세는지 정리한 뒤에야 전후 결과를 비교할 수 있었다.

같은 PC의 4인 실행에서 fallback이 줄었다

Host 1개와 Guest 3개를 같은 PC에서 실행한 수동 비교 결과다. 수정 전후를 별도로 실행해 여러 프로세스의 CPU 경쟁과 창 포커스, 조작 시점 차이가 포함될 수 있다.

로컬 계측 PC · 실험 당시와 동일하게 유지한 환경

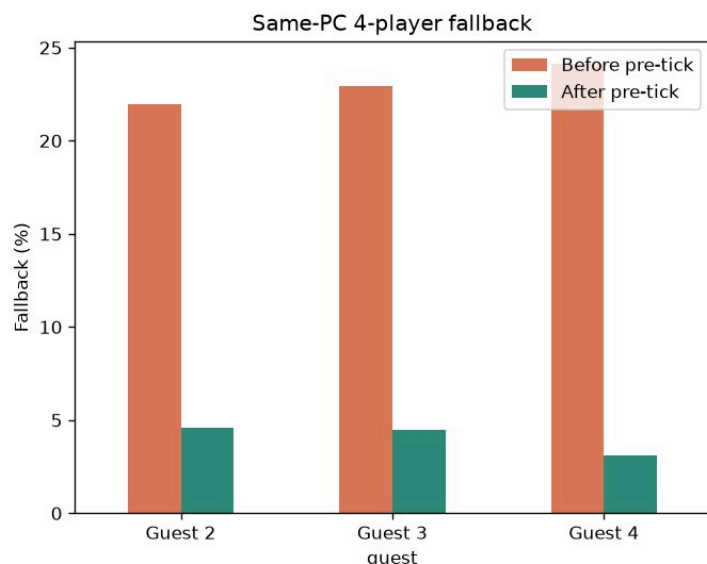
항목	구성
CPU	Intel Core Ultra 7 155H · 16코어 / 22논리 프로세서
GPU	NVIDIA GeForce RTX 4050 Laptop GPU
메모리	약 32GB · OS 인식 31.51GiB
OS	Windows 11 Enterprise
Unity	6000.3.22f1
호스트 실행 정책	목표 60 FPS · VSync OFF · 백그라운드 실행 유지

위 사양은 로컬 호스트 PC 기준이다. 목표 FPS는 실행 설정이며 실제 프레임 간격은 별도 계측값으로 비교했다.

4인 비교에 사용한 `fallback-4player` 빌드의 저장 설정은 창 모드 960×540, 그래픽 프리셋 보통, VSync OFF다. 뒤에서 다루는 2PC 입력 실험의 로컬 호스트 빌드는 창 모드 2880×1800, 보통, VSync OFF로 저장돼 있다. 같은 PC의 전후 비교와 별도 2PC 결과는 이 실행 조건을 나눠 기록했다.

Guest Entity	수정 전 fallback	수정 후 fallback	수정 후 실제 입력 소비
2	21.95%	4.57%	95.43%
3	22.94%	4.45%	95.55%
4	24.14%	3.09%	96.91%

UDP 패킷은 도착했는데 왜 입력은 누락됐을까? — Host Tick Deadline 추적 · 계속



개발 당시 입력 비교 기록. 본문의 소비율은 위 표의 Resolve 결과 기준으로 구분했다.

세 Guest 모두 fallback 비율이 낮아졌다. 이 결과와 함께 입력 큐에 대기한 상태에서 Resolve를 먼저 실행하던 경로가 바뀌었는지 확인했다. 관측된 수치 변화와 실행 순서의 변화를 함께 봐야, 단순히 이번 실행의 네트워크 상태가 좋았던 것인지에 대한 설명을 보강할 수 있었다.

느린 Pump를 열어 보니 계측 비용이 컸다

개선 후 Profiler에서는 Pump 구간이 크게 보이는 프레임이 있었다. 큐에서 입력을 꺼내 삽입하는 코드 자체가 느린지, 그 안에서 함께 수행하는 진단이 느린지 분리했다. 9월 19일 기록의 느린 프레임 2209에서 Pump는 23.2457ms였지만 실제 SubmitInput은 0.0483ms였다.

Profiler 관측 프레임	Pump	SubmitInput	Diagnostics 비중	할당량
느린 프레임	23.2457ms	0.0483ms	98.26%	161,303B
비교 프레임	5.2627ms	0.0248ms	92.21%	246,927B

상세 payload를 해석하고 기록하는 진단 경로가 대부분을 차지했다. 평상시에는 상세 기록을 끄고 1초 요약을 사용하도록 분리했다. 문제가 재현될 때 필요한 범위의 추적만 켤 수 있게 해, 측정 코드가 측정하려던 실행 순서를 다시 흔드는 일을 줄였다.

할당량도 같이 봤지만 GC.Alloc만으로 해당 프레임에 GC 수거가 발생했다고 결론 내리지는 않았다. 비교 프레임의 할당량이 더 컸는데 Pump 시간은 짧았다. 할당, 문자열 처리, 진단 함수 CPU 시간, 실제 GC pause를 나눠 확인해야 했다. 이 과정에서 큰 부모 마커 하나만 보고 입력 처리 알고리즘을 바꾸는 접근을 피할 수 있었다.

2PC에서 남은 deadline miss를 다시 봤다

9월 25일 별도 실험에서는 PC A에 Host 1개, PC B에 Guest 3개를 실행했다. 분석 대상으로 잡은 경과 시간 30~205초의 완결 계측 창은 약 173.936초였다. 이 구간의 실제 입력 소비는 합계 30,633 / 31,308 Tick, **97.8440%**였다. 같은 PC의 전후 실험과 실행 환경·관측 구간이 다르므로 하나의 연속된 개선율로 연결하지 않았다.

UDP 패킷은 도착했는데 왜 입력은 누락됐을까? — Host Tick Deadline 추적 · 계속

Guest Entity	실제 입력 / Resolve	소비율
2	10,206 / 10,436	97.7961%
3	10,208 / 10,436	97.8153%
4	10,219 / 10,436	97.9207%

남은 입력을 추적하니, 최초 삽입 시도 기준으로 계속 수신 경계에서 이미 stale인 비율이 약 2.0222~2.1752%였다. 반면 수신 시점에는 유효했지만 삽입할 때 stale이 된 비율은 0.0096~0.0288%였다. 이 비율의 분모는 Resolve가 아닌 최초 삽입 시도이므로 위 표의 fallback과 직접 더하지 않았다.

이 실행에서는 호스트 큐 이후의 대기보다 수신 경계 이전의 경로를 더 살펴볼 필요가 있었다. 그 앞에는 Guest의 입력 생성과 송신 요청, OS와 네트워크, C++ 중계 서버가 함께 있다. 관측 경계를 기준으로 후보를 줄일 수 있었지만, 그중 어느 하나가 모든 잔여 miss의 원인이라고 확정할 자료는 부족했다.

이후 확인할 기준

이번 수정의 핵심은 큐 처리 위치를 Tick의 소비 deadline과 맞춘 것이다. 입력을 많이 보내는 것만큼, 실제 시뮬레이션이 그 입력을 언제 볼 수 있는지가 중요했다. 수신·삽입·Resolve를 나눈 계측 덕분에 개선 후 남은 문제도 같은 원인으로 뭉뚱그리지 않고 다시 조사할 수 있었다.

실험 장비와 설정은 그대로 유지했다. 다음 비교에서는 전원 상태와 창 포커스까지 실행 시작 시점에 기록하고, 같은 입력을 재생해 수동 조작의 차이를 줄이려 한다. 평균 소비율이 높더라도 드문 hitch나 단발 입력의 deadline miss가 남을 수 있어, 점프는 별도의 이벤트 이력으로 검증했다.

09 / Troubleshooting

예측에서는 점프했는데 호스트는 왜 거절했을까? — 단발 입력 동기화 문제 해결

이동 상태와 점프 Down을 구분하고, 캡처한 edge가 호스트의 최종 결정에 연결되는지 InputTick별로 추적했다.

[블로그 원문 ↗](#)

문제 Guest에서 예측한 점프가 다음 호스트 스냅샷에서 지상으로 되돌아갔다.

원인 단발 입력이 목표 Tick의 결정 전에 반영되지 않으면 점프 판정이 사라졌다.

해결 Jump edge를 새 InputTick에 부착할 때 최초 1회 즉시 전송했다.

결과 3 Guest 재실험의 점프 369건을 호스트 삽입과 연결했고, 362건 수락·7건 stale을 확인했다.

역할 점프 lifecycle 계측, 최초 송신 경로 수정, Guest-Host 로그 대조를 담당했다.

용어 정리

용어	뜻
Tick	호스트가 물리 계산을 한 번 하는 단위(초당 60번)
InputTick	이 입력이 실행될 목표 Tick 번호
edge	버튼을 새로 누른 그 순간에만 한 번 생기는 입력
Prediction	서버 응답을 기다리지 않고 내 화면에서 먼저 움직여 보여주는 것
Reconciliation	내가 예측한 위치를 호스트가 확정한 값에 맞춰 다시 바로잡는 과정
fallback	그 순간 받은 입력이 없어서 이전 값을 대신 쓰는 처리
deadline	그 Tick이 입력을 받아줄 수 있는 마감 시점
datagram	UDP로 한 번에 보내는 패킷 한 덩어리
LastResolvedInputTick	호스트가 “여기까지는 확정했다”고 알려주는 값
RejectedStale	이미 그 Tick 판정이 끝나서 뒤늦게 도착한 입력이 거절되는 것
RejectedNotGrounded	착지 상태가 아니라서 점프가 거절되는 것
Pre-Tick Pump	호스트가 Tick을 확정하기 직전에 이미 도착한 입력을 먼저 반영해두는 처리

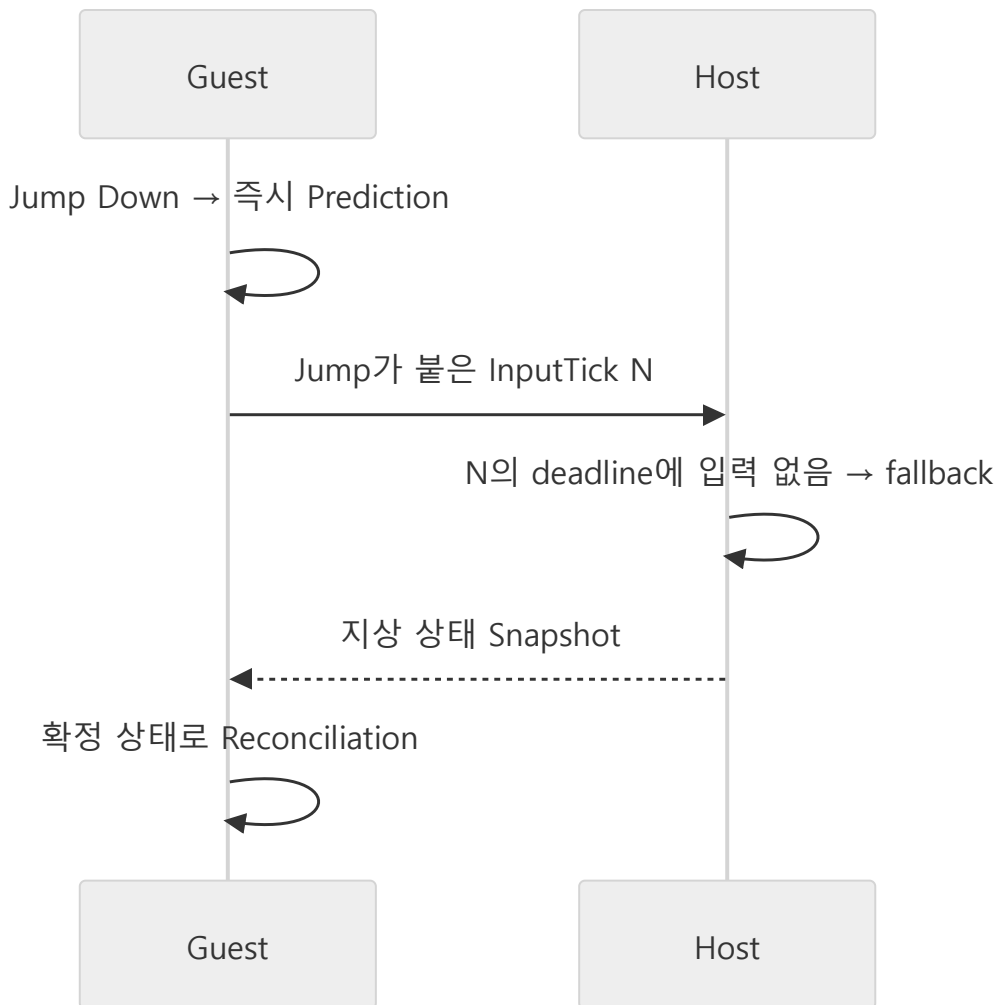
예측에서는 점프했는데 호스트는 왜 거절했을까? — 단발 입력 동기화 문제 해결 · 계속

점프했다가 다시 바닥으로 돌아왔다

Guest에서 점프 키를 누르면 로컬 Prediction이 바로 상승을 시작했다. 그런데 잠시 후 캐릭터가 지상으로 되돌아오는 경우가 있었다. 화면만 보면 점프 애니메이션이 취소되거나 물리 보정이 과도하게 적용된 것처럼 보였다.

개선 전 녹화. 오른쪽 아래 화면에서 호스트가 점프를 확정하지 못해 거절하고, 예측 위치에서 곧바로 확정 위치(지상)를 받아 바닥으로 쫓히는 장면이다. [영상 파일 열기](#)

호스트 기록을 따라가 보니, Guest가 점프를 예측한 입력 Tick을 호스트에서는 fallback으로 처리할 수 있었다. 호스트의 확정 상태에는 점프가 없고, Guest는 이후 그 상태를 받아 Reconciliation을 수행한다. 화면의 복귀는 마지막에 드러난 결과였다. 그 앞에서 점프 입력이 어디까지 전달됐는지를 먼저 확인해야 했다.



이동은 유지되지만 Down은 한 번 발생한다

앞으로 이동하는 입력은 여러 Tick 동안 같은 상태가 유지된다. 한 Tick에서 실제 입력을 사용하지 못해도 Held fallback이 직전 방향을 유지하면 이동이 이어질 수 있다. 물론 방향 전환이나 정지 시점에는 이 차이도 오차로 나타난다.

반면 Jump Down은 버튼을 새로 누른 그 순간에만 한 번 발생하는 edge다. 이 한 번을 놓쳤다고 다음 Tick에도 계속 “눌렀다”고 복제하면 점프가 여러 번 반복될 수 있다. 반대로 다음 Tick부터 바로 지우면, 호스트가 하필 그 Tick을 놓쳤을 때 점프 자체가 일어나지 않는다.

예측에서는 점프했는데 호스트는 왜 거절했을까? — 단발 입력 동기화 문제 해결 · 계속

렌더 프레임에서 캡처한 edge를 어느 InputTick에 붙일지도 중요했다. 그 시점의 command가 이미 생성됐다면 같은 Tick을 뒤늦게 덮어쓰기보다, 다음으로 사용할 수 있는 새 command까지 edge를 유지해야 한다. 단순히 버튼을 읽었는지 기록하는 것만으로는 이 연결이 정상인지 확인할 수 없었다.

다섯 단계에 같은 입력 키를 남겼다

점프를 다음 다섯 경계로 나눴다. Guest와 Host의 로그는 (`room`, `world`, `authority`, `entity`, `inputTick`)으로 연결했다. 서로 다른 PC의 monotonic 시계 값은 직접 빼지 않았다. 프로세스별 시간의 시작점이 달라, 그 차이를 전송 지연으로 해석하면 잘못된 결과가 나온다.

단계	확인하려던 내용
Captured	Guest가 Jump Down을 읽었는가
Attached	edge가 실제 새 InputTick의 command에 연결됐는가
FirstSendRequested	그 command의 최초 UDP 송신을 요청했는가
Inserted	호스트가 해당 Tick을 버퍼에 수락했는가
Resolved	호스트가 입력을 처리하고 어떤 점프 결정을 내렸는가

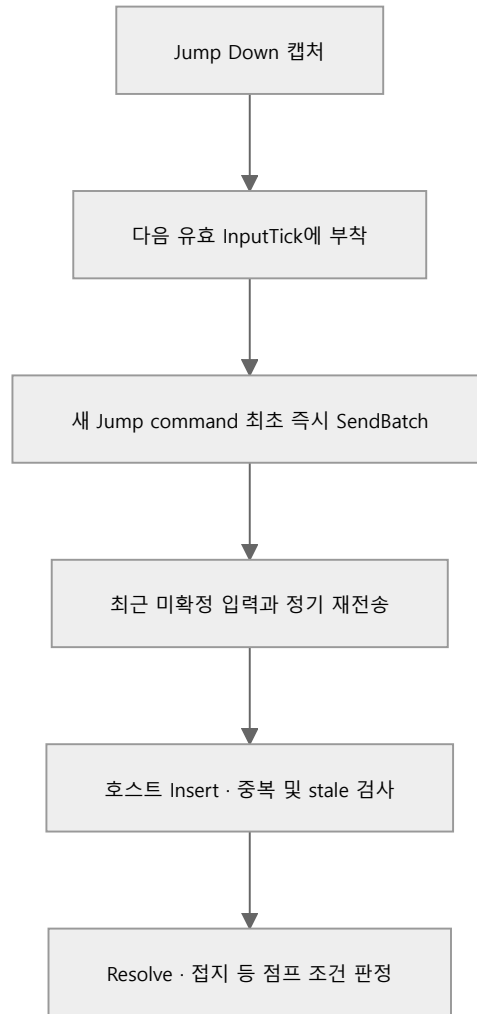
`FirstSendRequested`는 UDP SendAsync 호출을 시작하는 경계다. 소켓의 물리 송신 완료나 상대의 수신 응답까지 확인한 시점으로 확대하지 않았다. `Inserted`도 최종 점프 성공과 구분했다. 입력을 수락한 뒤에도 접지 조건이 맞지 않으면 게임 규칙상 점프가 거절될 수 있다.

이렇게 나누자 동일한 “점프가 안 됨” 안에 서로 다른 사건이 있음을 확인할 수 있었다. edge가 command에 붙지 않은 경우, 송신 요청이 늦은 경우, 도착했지만 deadline을 지난 경우, 입력을 처리했지만 접지 조건이 맞지 않은 경우는 각각 다른 수정이 필요했다.

새 점프 command의 첫 전송을 앞당겼다

일반 이동 입력은 주기적인 batch로 묶어 전송한다. 30Hz 송신 주기라면 새 입력이 만들어진 뒤 다음 정기 송신까지 대기할 수 있다. Jump edge가 새 InputTick에 붙는 순간에는 이 대기를 추가하지 않도록 최초 1회 즉시 송신을 요청했다.

예측에서는 점프했는데 호스트는 왜 거절했을까? — 단발 입력 동기화 문제 해결 · 계속



최초 송신과 정기 재전송은 역할을 나눴다. 첫 전송은 command 생성 후 대기 시간을 줄이고, 최근 미확정 입력 최대 8개를 포함하는 재전송은 개별 datagram이 사라질 때 다시 전달될 기회를 만든다. 호스트는 동일한 Tick을 중복 적용하지 않도록 구분하고, Guest는 LastResolvedInputTick에 따라 완료된 이력을 정리한다.

이 재전송 방식에는 한 가지 짚어야 할 점이 있다 — RejectedStale이 하나 나왔다고 해서 그 점프가 실제로 실패했다는 뜻은 아니다. 첫 전송으로 점프가 이미 성공했더라도, Guest가 아직 LastResolvedInputTick을 못 받은 사이에는 같은 명령이 미확정 history에 남아 재전송된다. 호스트는 그 Tick을 이미 확정했으므로 이 재전송분을 RejectedStale로 거절한다. 즉 “이미 성공한 점프의 뒤늦은 사본이 거절된 경우”와 “진짜로 제때 못 들어가서 실패한 경우”가 같은 RejectedStale 안에 섞여 있을 수 있다. 실제로 수동 플레이 중에는 점프가 눈에 띄게 되돌아가는 느낌을 받지 못했는데, 이 구분과도 맞아떨어진다.

수정 후 전체 플레이 녹화(3분 40초). 점프 입력 전송 경로를 수정한 뒤 실제 플레이를 이어서 확인한 장면이다. [영상 파일 열기](#)

호스트가 fallback으로 그냥 Tick을 끝내도, “여기까지 끝났다”는 확정 신호는 앞으로 나아간다. 완료된 Tick의 판정은 확정되므로 첫 전송을 deadline 안에 넣는 것이 중요했다. 최초 즉시 송신과 [Pre-Tick Pump](#)가 각각 Guest의 대기과 Host의 반영 순서를 맡았다.

예측에서는 점프했는데 호스트는 왜 거절했을까? — 단발 입력 동기화 문제 해결 · 계속

검증: 입력을 수락했는지, 점프했는지 따로 켜다

9월 25일 재실험에서는 Host에 연결된 Guest를 1개, 2개, 3개로 늘리며 실행 로그를 분리했다. 실행 길이와 수동 조작 시점이 달라 Guest 수에 따른 성능 곡선으로 사용하지 않았다. 목적은 캡처한 특정 입력을 호스트의 최종 결정까지 대조하는 것이었다.

로컬 계측 PC · 실험 당시와 동일하게 유지한 환경

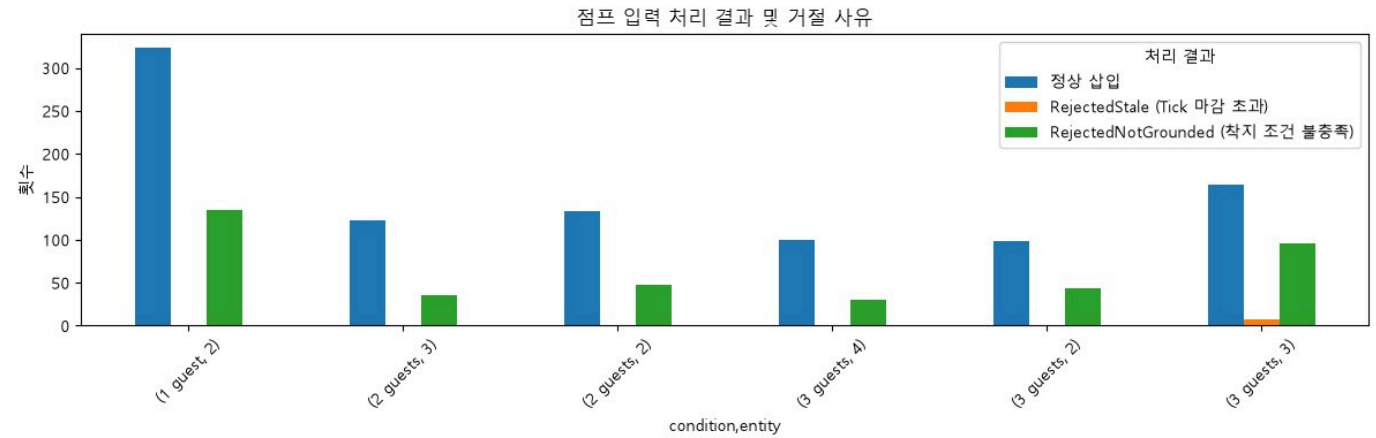
항목	구성
CPU	Intel Core Ultra 7 155H · 16코어 / 22논리 프로세서
GPU	NVIDIA GeForce RTX 4050 Laptop GPU
메모리	약 32GB · OS 인식 31.51GiB
OS	Windows 11 Enterprise
Unity	6000.3.22f1
호스트 실행 정책	목표 60 FPS · VSync OFF · 백그라운드 실행 유지

위 사양은 로컬 호스트 PC 기준이다. 목표 FPS는 실행 설정이며 실제 프레임 간격은 별도 계측값으로 비교했다.

로컬 점프 재실험 빌드의 저장 설정은 창 모드 1280×720, 그래픽 프리셋 보통, VSync OFF다. 호스트 로그에서도 Unity 6000.3.22f1과 RTX 4050 Laptop GPU의 Direct3D 11 실행을 확인했다.

Guest 수 / Entity	Attached	삽입 매칭	Accepted	stale	Executed	NotGrounded
1 / 2	323	323	323	0	189	134
2 / 3	123	123	123	0	88	35
2 / 2	133	133	133	0	86	47
3 / 4	100	100	100	0	70	30
3 / 2	98	98	98	0	54	44
3 / 3	171	171	164	7	68	96

예측에서는 점프했는데 호스트는 왜 거절했을까? — 단발 입력 동기화 문제 해결 · 계속



위 표를 그대로 그래프로 그린 것이다. *RejectedStale*(주황)은 (3 guests, 3) 조건에서만 살짝 보일 만큼 적고, *RejectedNotGrounded*(초록)이 거절 대부분을 차지한다.

이 표는 Guest의 Attached Tick을 기준으로 연결한 결과다. 서로 다른 Entity와 반복 판정이 섞인 전체 로그 행 수를 나눠 성공률을 계산하지 않았다. 3 Guest 실행에서는 369개 입력이 모두 호스트 삽입 기록에 연결됐고, 그중 362개는 수락 후 Resolved 결정까지 이어졌다. 192개가 실제 점프를 실행했고 170개는 접지 조건에서 거절됐다.

따라서 362건 수락을 362회 점프 성공이라고 적을 수는 없다. 반대로 NotGrounded 170건을 패킷 유실로 세어서도 안 된다. 공중에서 추가로 누른 점프처럼 유효하게 도착해도 실행 조건을 만족하지 않는 입력이 포함된다.

송신 경로의 변경은 같은 Guest 시계 안에서도 확인했다. 3 Guest의 Attached→FirstSendRequested는 각각 p95 1ms였고, 최대 1~2ms였다. 반면 Captured→Attached의 최대는 Entity별 14~78ms였다. command가 만들어진 뒤의 대기는 짧아졌지만, edge가 다음 유효 Tick에 불기까지의 지연은 별도로 남을 수 있었다.

남은 7건은 어느 경계에서 늦었나

stale로 거절된 7건은 전부 Entity 3에서 나왔고, 모두 `queuedBeforeResolve=0`이었다. 이 건 호스트가 큐에서 이 입력을 확인했을 때 그 Tick의 판정이 이미 끝나 있었다는 뜻이다. 실제로 호스트가 그 Tick 판정을 마친 시각은, 우리가 “입력을 받았다”고 기록한 시각보다 1.6~12.2ms 더 빨랐다. 즉 “큐에서 기다리다가 놓친 것”과는 다른 패턴이었다.

입력이 버퍼에 들어가기 직전까지 4.3~14.7ms를 기다리긴 했지만, 그 사이에 목표 Tick을 넘겨버리는 새 판정이 일어난 건 아니었다. 입력은 “도착했다”고 기록한 그 순간부터 이미 늦어 있었다. 그래서 Pre-Tick Pump를 더 자주 실행한다고 이 7건이 되살아나지는 않을 거라고 봤다.

이 7건은 버튼을 누른 순간부터 명령에 불기까지 0~27ms, 그 뒤 전송 요청까지는 0~1ms 걸렸다. 그래서 의심할 곳이 두 군데로 좁혀졌다 — edge가 명령에 불기까지 기다린 시간, 그리고 그 이후의 송신-중계-수신 경로다. 같은 실행에서 Guest 화면이 크게 멈춘 구간이 있긴 했지만, 그게 정확히 이 7건과 같은 순간에 일어났다는 증거는 없어서 “프레임 끊김 때문”이라고 단정하지는 않았다.

단발 입력은 평균 비율만으로 확인하기 어려웠다

이동 입력이 99% 넘게 잘 도착해도, 점프 버튼 딱 한 번이 제시간에 못 들어가면 플레이어는 바로 눈치챈다. 그래서 이번에는 “평균적으로 입력이 얼마나 잘 갔나”만 보지 않고, 사건 하나하나가 언제 명령이 됐고 어떤 판정을 받았는지 끝까지 따라갔다. 화면에서 보이는 “점프가 안 됨”을 입력이 아예 안 들어간 경우와 게임 규칙상 거절된 경우로 나눠서 볼 수 있게 된 게, 이번에 얻은 가장 큰 변화였다.

예측에서는 점프했는데 호스트는 왜 거절했을까? — 단발 입력 동기화 문제 해결 · 계속

다음 검증에서는 입력을 똑같이 재생하고 접지 상태도 고정해서, 같은 점프 시퀀스가 같은 조건에서 매번 어떻게 판정되는지 비교해볼 필요가 있다. 지금 로그는 최초 전송 경로와 이번에 놓친 7건을 설명해주지만, 어떤 지연 상황에서도 점프 누락이 절대 안 생긴다고 보장해주지는 않는다. 그래도 비슷한 사례가 또 나오면 같은 방식(room, world, authority, entity, inputTick)으로 계속 연결해서 추적할 수 있도록 계속 기준은 그대로 남겨뒀다.

10 / Client Framework

공통 서비스와 씬 객체의 수명을 나눈 클라이언트 구조

앱 전체에서 유지할 서비스와 씬에서 생성·정리할 객체를 구분했다. 공통 서비스의 조립과 보관을 나누고 씬 진입점에서 필요한 객체를 연결했다.

[블로그 원문 ↗](#)

공통 서비스와 씬 객체의 수명은 다르다

로그인 정보, 통신, 방 상태, UI 관리 기능은 씬 전환 이후에도 필요하다. 캐릭터, 씬의 UI, 카메라 연결과 이벤트 구독은 해당 씬이 끝나면 정리해야 한다. 이 두 수명을 구분하고, 객체를 만들고 정리할 주체를 정했다.

프레임워크 디렉터리 구조

아래는 클라이언트의 주요 디렉터리와 대표 파일이다.

TEXT

```
Client/Assets/  
├─ Game/                # 프로젝트 코드와 자체 구성 자산  
├─ ThirdParty/          # 외부 에셋  
└─ ThirdPartyUsed/      # 사용 외부 에셋
```

프로젝트 코드와 외부 에셋을 별도 디렉터리로 구분했다. Game 내부의 주요 구조는 다음과 같다.

TEXT

```
Client/Assets/Game/  
├─ App/  
│   └─ Runtime/  
│       ├── Bootstrap/  
│       │   ├── GameInstance.cs  
│       │   ├── AppCompositionRoot.cs  
│       │   └─ AppServices.cs  
│       └─ EntryPoints/  
│           ├── MainMenuEntryPoint.cs  
│           ├── WaitingRoomEntryPoint.cs  
│           ├── LoadingSceneEntryPoint.cs  
│           └─ InGameEntryPoint.cs  
├─ Core/  
│   └─ StateMachines/  
├─ Infrastructure/  
│   ├── AssetManagement/  
│   │   ├── AssetManager.cs  
│   │   ├── PrefabFactory.cs  
│   │   └─ Catalogs/  
│   │       └─ Pooling/  
│   ├── SceneLoading/  
│   ├── Input/  
│   └─ Network/  
└─ Features/
```

공통 서비스와 씬 객체의 수명을 나눈 클라이언트 구조 · 계속

TEXT · 계속

```

|   └─ Room/
|   └─ Match/
|   └─ Player/
|   └─ PlayerMovement/
└─ UI/
    └─ Runtime/Core/
        └─ Base/
        └─ Layers/
        └─ Management/

```

영역	책임
App	앱 초기화, 서비스 연결, 씬 진입·종료 처리
Core	캐릭터 상태 계약과 공통 상태 머신
Infrastructure	자산 접근, 객체 생성·풀링, 씬 로딩, 입력과 통신 기반
Features	방, 게임 진행, 캐릭터와 이동 등 기능별 상태와 동작
UI	화면의 공통 수명과 표시 계층 관리

조립과 보관을 분리했다

TEXT

GameInstance

- AppCompositionRoot에서 서비스 조립
- AppServices에 서비스 참조 보관
- 씬 EntryPoint에서 씬 객체와 연결

구성 요소	맡은 일
GameInstance	공통 진입점, 초기화, 프레임 갱신, 앱 종료
AppCompositionRoot	의존 관계에 맞춰 서비스를 만들고 생성자로 연결
AppServices	공통 서비스 참조 보관과 종료 관리
씬 EntryPoint	UI 등록, 캐릭터·카메라 연결, 씬 구독과 풀 범위 정리

GameInstance에는 싱글턴을 적용했다. 중복 인스턴스를 제거하고 씬 전환 후에도 유지한다. 초기화 가드는 초기화 중 재접근으로 서비스가 중복 생성되는 것을 막는다.

서비스 조립부는 필요한 객체를 생성자로 전달한다. 어떤 서비스가 무엇에 의존하는지 조립 코드에 드러나는 수동 의존성 주입 구조다. 일부 서비스는 서비스 컨테이너의 생성자에서도 생성한다.

GameInstance가 서비스를 조회하는 진입점이고 AppServices가 서비스 참조를 보관한다. 두 객체는 개별 서비스와 동일한 인터페이스로 요청을 대신 처리하는 프록시 구조로 구성하지 않았다.

공통 서비스와 씬 객체의 수명을 나눈 클라이언트 구조 · 계속

씬 진입점이 씬의 연결과 정리를 맡는다

게임 씬 진입점은 풀 범위를 만들고 캐릭터를 대여한 뒤 카메라와 게임 기능을 연결한다. 씬 종료 시 진입점이 이벤트 구독과 전환 작업을 정리하고, 캐릭터를 반납한 뒤 풀 범위를 종료한다.

TEXT

씬 진입

- 씬 UI 등록
- 풀 범위 생성과 캐릭터 대여
- 기능 연결과 이벤트 구독

씬 종료

- 구독과 전환 작업 정리
- 씬 UI 등록 해제
- 캐릭터 반납과 풀 범위 종료

앱 종료와 씬 파괴 순서는 달라질 수 있다. 씬 진입점은 풀 범위가 이미 종료됐는지 확인해 정리한다. 앱 종료 시 서비스 컨테이너는 남은 서비스와 풀을 정리한다.

씬 전환은 로딩 기능과 나뉘었다

TEXT

```
Infrastructure/SceneLoading/  
├─ ISceneLoader.cs  
├─ UnitySceneLoader.cs  
├─ SceneFlowManager.cs  
└─ SceneCatalog.cs
```

씬 전환 관리자는 목적 씬과 전환 진행 상태를 보관한다. 실제 Unity 로딩 API는 Loader 구현이 맡는다. Loading 씬을 거쳐 목적 씬을 비동기로 로딩하고 진행률을 전달한 뒤 활성화한다. 전환 중 중복 요청을 받으면 다시 전환하지 않는다.

Vivox 음성 채팅을 방과 로그인 수명에 연결했다

Vivox 음성 채팅을 연동하고, 음성 서비스가 게임 로그인과 방 상태 변경을 구독하도록 구성했다. 게임 서버 로그인 이후 플레이어 ID로 음성 로그인을 진행하고, 방 ID에 대응하는 음성 채널에 참가한다.

TEXT

게임 로그인

- Vivox 초기화·로그인
- 방 입장
- room_ + 방 ID에 대응하는 음성 채널 참가
- 방 이탈 시 채널 퇴장
- 게임 로그아웃 시 음성 로그아웃

공통 서비스와 씬 객체의 수명을 나눈 클라이언트 구조 · 계속

구성	책임
VoiceChatService	로그인·방 변경과 음성 연결 수명 조정
VivoxSdkGateway	Vivox SDK 호출과 참가자 이벤트 변환
VoiceChatSnapshot	음성 연결 상태, 채널, 마이크 음소거 상태 전달
VoiceParticipantSnapshot	참가자 ID, 발언·음소거 상태 전달
UI_RoomVoice	음성 상태와 방 참가자 표시

TEXT

```
Client/Assets/Game/
├─ Features/VoiceChat/Runtime/
│   └─ VoiceChatService.cs
│   └─ VivoxSdkGateway.cs
│   └─ VoiceChatContracts.cs
└─ UI/Derived/
    └─ UI_RoomVoice.cs
    └─ UI_RoomVoiceAccountRow.cs
```

음성 서비스는 비동기 로그인과 채널 전환을 한 번에 하나씩 진행하도록 조정한다. 같은 방 채널에 이미 참가했다면 다시 입장하지 않고, 다른 방으로 이동할 때는 기존 채널에서 나온 뒤 새 채널로 들어간다.

음성 서비스는 앱 수명으로 유지되므로 대기실에서 게임 씬으로 이동하는 것만으로 방 채널을 종료하지 않는다. 방 이탈은 채널을 정리하고, 서비스 종료는 로그인·방·SDK 이벤트 구독을 해제하고 진행 중 작업을 취소한다.

SDK의 참가자 정보는 게임에서 사용하는 상태 객체로 변환한다. 화면은 이 상태와 변경 이벤트를 사용해 표시하고, 마이크 음소거 요청은 음성 서비스를 통해 SDK에 전달한다.

음성 입력의 배경음 유입은 해결 과제로 남았다

상대방 마이크를 통해 배경 음악이 함께 들리는 문제가 있다. 음성 입력의 에코 제거·노이즈 억제·자동 게인 제어는 적용하지 않았으며, 다음 개선 과제로 정리했다.

개선 항목	목적
AEC — 에코 제거	각 참가자의 스피커 재생음이 자신의 마이크로 다시 들어가 상대방에게 되돌아가는 에코 감소
Noise Suppression — 노이즈 억제	마이크에 섞이는 지속적인 배경 소음 감소
AGC — 자동 게인 제어	참가자마다 다른 마이크 입력 음량 자동 조절

배경 음악 유입은 노이즈 억제를 우선 적용할 과제다. 음악은 음성과 섞여 들어올 수 있으므로 노이즈 억제로 완전히 제거된다고 전제하지 않는다. 에코 제거와 입력 음량 조절도 함께 조정하되, 말소리 손상 여부를 확인하며 적용한다.

공통 서비스와 씬 객체의 수명을 나눈 클라이언트 구조 · 계속

구조를 나눈 결과

공통 객체의 조립은 조립부에, 앱 전체의 서비스 참조와 종료는 컨테이너에 모았다. 씬 진입점은 씬에서 사용하는 객체의 연결과 정리를 맡는다. **객체의 수명이 끝나는 시점과 정리 주체를 함께 정의한 구조다.**

자산·폴 범위 구성은 [프리팹 조회·생성·재사용](#)에서, 메시지와 구독 전달은 [이벤트 전달 구조](#)에서 이어서 설명한다.

11 / Client Framework

프리팸 조회·생성·재사용의 책임을 나눈 자산 관리 구조

ScriptableObject Catalog로 프리팸을 조회하고 생성 공장에서 인스턴스를 만들었다. 풀은 객체를 대여·반납하고 씬별 범위에서 함께 정리하도록 구성했다.

[블로그 원문 ↗](#)

조회·생성·재사용을 각각의 책임으로 나눴다

프리팸 경로와 이름을 기능 코드마다 관리하는 대신 식별자로 원본을 찾도록 구성했다. 원본 조회, 인스턴스 생성, 풀에서의 재사용은 각각 다른 객체가 맡는다.

디렉터리와 파일 구조

TEXT

```
Client/Assets/Game/Infrastructure/AssetManagement/
├ AssetManager.cs           # 원본 프리팸 조회
├ PrefabFactory.cs         # 프리팸 인스턴스 생성
├ Catalogs/
│   ├── PrefabCatalog.cs   # 식별자 → 프리팸 매핑
│   ├── PrefabCatalogEntry.cs # 매핑 항목
│   └ PrefabCatalog.asset  # ScriptableObject 데이터
└ Pooling/
    ├── PoolManager.cs      # 풀 범위 생성·관리
    ├── PoolScope.cs        # 씬 범위의 풀 등록·정리
    ├── ObjectPool.cs       # 프리팸별 대여·반납
    ├── PoolableObject.cs   # 객체의 풀 소유·대여 상태
    ├── PoolRegistrationInfo.cs # 풀 등록 설정
    └ PoolInfo.cs           # 풀 상태 정보
```

Catalog는 식별자와 원본을 연결한다

ScriptableObject Catalog에 프리팸 식별자와 GameObject 참조를 정의했다. Catalog는 런타임 조회용 Dictionary를 구성하고 빈 식별자, 중복 정의, 누락된 프리팸을 검사한다.

TEXT

```
ePrefabId
→ AssetManager
→ PrefabCatalog의 Dictionary 조회
→ 원본 프리팸
```

여기서 Dictionary는 식별자와 프리팸을 연결하는 Map이다. 게임 월드의 지형 설정을 담은 MapLayoutConfig와는 역할이 다르다. 자산 접근은 직접 프리팸 참조를 사용한다.

프리팸 조회·생성·재사용의 책임을 나눈 자산 관리 구조 · 계속

생성 공장은 복제를 담당한다

PrefabFactory는 자산 관리자에서 원본을 받아 위치·회전·부모 조건에 따라 복제한다. 원본을 바꾸는 작업은 Catalog에서, 생성 동작을 바꾸는 작업은 공장에서 처리한다.

프리팸 원본으로 인스턴스를 복제하는 동작은 Prototype의 성격을 갖는다. 구현은 Unity의 복제 API를 사용한다. 생성 책임을 공장에 모았으며, 하위 클래스가 생성 메서드를 재정의하는 Factory Method 구성과는 구분된다.

풀은 대여·반납을 담당한다

반복적인 객체 생성·제거와 메모리 할당 부담을 줄이고, GC가 누적된 회수 대상을 한꺼번에 정리하면서 화면이 멈추는 현상을 예방하기 위해 오브젝트 풀링을 적용했다. 사용이 끝난 객체는 보관 한도 안에서 비활성화해 반납하고, 다음 요청에서 재사용하도록 구성했다.

재사용으로 GC 부담을 줄인다

객체 생성 과정에서 관리 메모리 할당이 반복되면 GC가 회수할 대상도 늘어날 수 있다. GC가 많은 작업을 한 번에 수행하면 프레임이 지연되어 화면이 잠시 멈춘 것처럼 보일 수 있다. 풀링은 기존 객체를 재사용해 반복 할당과 생성·제거 부담을 줄이는 방법이다. [Unity 오브젝트 재사용 문서](#)

TEXT

생성·제거를 반복하는 방식

- 사용할 때 생성
- 사용이 끝나면 제거
- 다음 사용에서 다시 생성

풀링 방식

- 풀에서 대여
- 사용이 끝나면 비활성화해 반납
- 다음 사용에서 같은 객체 재사용

객체 제거 비용과 C# GC의 메모리 회수 비용은 구분된다. 풀링은 두 부담을 줄이는 데 도움이 되지만, 사용 중 생성하는 문자열·임시 컬렉션 등의 할당까지 없애지는 않는다.

대여와 반납 흐름

TEXT

씬 EntryPoint

- PoolManager에서 PoolScope 생성
- 프리팸별 ObjectPool 등록
- 객체 대여
 - └ 대기 객체가 있으면 재사용
 - └ 필요하면 PrefabFactory로 생성
- 객체 사용
- 원래 풀로 반납

프리팸 조회·생성·재사용의 책임을 나눈 자산 관리 구조 · 계속

동작	처리
대여	대기 객체 선택 또는 확장, 부모·Transform 정리, 활성화, 대여 알림
반납	풀 소유·대여 상태 확인, 반납 알림, 대기 객체로 보관
보관 한도 초과	초과한 반납 객체 제거
범위 종료	범위에 속한 풀과 추적 객체 정리

ObjectPool은 사용 가능한 객체를 Queue에 보관하고 자신이 생성한 전체 객체를 따로 추적한다. 다른 풀의 객체를 반납하거나 같은 객체를 두 번 반납하는 동작은 소유·대여 상태로 검사한다.

최대 보관 수는 대기 객체의 한도다. 동시에 대여할 수 있는 객체 수의 절대 상한을 뜻하지 않는다.

썬별 범위에서 정리한다

PoolScope는 썬 범위의 프리팸별 풀과 공통 부모 객체를 소유한다. 썬 종료 시 진입점이 범위를 종료하고, 범위는 대기 객체와 대여 객체를 포함해 자신이 추적하는 객체를 정리한다. 앱 종료 시 PoolManager가 남은 범위를 종료한다.

평소 반납에서는 객체를 재사용하지만, 보관 한도를 넘거나 썬의 풀 범위를 종료할 때는 객체를 제거한다. **많은 객체를 한꺼번에 정리하는 시점에는 제거 비용이 집중될 수 있다.** 풀링은 반복 생성·제거를 줄이는 구조이며, 일괄 정리 비용까지 자동으로 분산하는 구조는 아니다.

변경할 내용	담당 파일
식별자에 연결할 프리팸	PrefabCatalog와 데이터
원본 조회 규칙	AssetManager
복제 생성 조건	PrefabFactory
대여·반납과 확장 규칙	ObjectPool
썬 단위 등록과 종료	PoolScope
전체 풀 범위 관리	PoolManager

조회·생성·재사용·종료의 변경 지점을 나눈 것이 이 구조의 핵심이다.

12 / Client Framework

UI의 표시 순서와 공통 수명을 관리하는 구조

UIManager에서 화면 등록과 표시 순서를 관리하고 UIBase에서 초기화·열기·닫기의 공통 순서를 유지했다. 화면별 동작은 흑으로 확장하고 ESC 입력에는 처리 우선순위를 뒀다.

[블로그 원문 ↗](#)

화면의 공통 동작과 개별 동작을 나눴다

게임 화면, 팝업, 모달은 표시 우선순위와 닫기 정책이 다르다. UI 관리자는 화면 등록과 순서를 맡고, 공통 베이스는 화면의 초기화·열기·닫기를 맡도록 구성했다.

디렉터리와 파일 구조

TEXT

```
Client/Assets/Game/UI/Runtime/Core/
├─ Base/
│   ├── UIBase.cs           # 공통 수명과 바인딩
│   └─ eUILayer.cs         # 표시 계층 정의
├─ Management/
│   ├── UIManager.cs       # 등록·조회·열기·닫기·순서
│   └─ UIRoot.cs           # 계층별 부모 참조
├─ Layers/
│   ├── UIFixed.cs
│   ├── UIWindow.cs
│   ├── UIPopup.cs
│   ├── UIModal.cs
│   ├── UIToast.cs
│   └─ UISystem.cs
└─ Components/
    ├── UIDraggable.cs
    └─ UIOptionSelector.cs
```

타입으로 화면을 찾고 Layer에 배치한다

UIManager는 화면 타입과 UI 객체를 연결한다. 호출자는 화면 타입으로 열기·닫기·조회를 요청한다. 관리자는 화면의 Layer에 맞는 UIRoot 부모를 선택하고 표시 순서를 관리한다.

TEXT

```
UIRoot
├─ Canvas
│   ├── FixedRoot
│   ├── WindowRoot
│   ├── PopupRoot
│   ├── EscapeMenuRoot
│   ├── ModalRoot
│   ├── ToastRoot
│   └─ SystemRoot
```

UI의 표시 순서와 공통 수명을 관리하는 구조 · 계속

Root	구분한 역할
FixedRoot	HUD·조준점처럼 기본 화면에 고정되는 UI
WindowRoot	포커스와 앞뒤 순서가 필요한 일반 화면
PopupRoot	팝업 표시
EscapeMenuRoot	ESC 메뉴와 메뉴에서 여는 설정 화면
ModalRoot	확인·취소 등 응답을 받는 모달
ToastRoot	짧은 알림
SystemRoot	로딩·페이드 등 최상단 시스템 화면

UIBase는 모든 화면의 공통 클래스이고, **FixedRoot**는 기본 고정 화면의 표시 계층이다. 공통 클래스의 상속 구조와 Canvas 아래의 표시 계층을 구분했다.

계층별 z-order와 창 의 포커스를 관리한다

Root 순서로 계층 사이의 앞뒤 관계를 나누고, Window·Popup·Modal은 각 계층 안의 표시 순서를 별도로 관리한다. 관리자는 순서 목록을 아래에서 위 순서로 보관하고, 목록 순서대로 자식 객체를 마지막 형제로 이동해 실제 표시 순서에 반영한다.

TEXT

- 창 열기 또는 열린 창 클릭
 - 해당 창을 순서 목록에서 제거
 - 목록의 마지막에 추가
 - 목록 순서로 Sibling 순서 갱신
 - 해당 계층의 최상단에 표시

UIWindow는 클릭을 받으면 포커스를 요청한다. **실제 z-order 변경은 UIManager가 담당한다.** 이미 열린 창을 다시 열 때도 목록에 중복으로 넣지 않고 최상단으로 옮긴다. 창을 닫으면 해당 항목을 제거하고 남은 순서를 갱신한다.

Window의 순서 목록과 Popup·Modal의 스택은 서로 분리했다. 관리자는 각 목록의 마지막 항목으로 최상단 화면을 찾는다.

UIBase의 상속 구조

TEXT

- UIBase # 초기화·열기·닫기·바인딩
- └─ UIFixed # 고정 화면
- └─ UIWindow # 포커스 요청과 ESC 확장
 - └─ UIDraggable # 드래그 이동
- └─ UIPopup # 팝업 닫기 정책 데이터
- └─ UIModal # 모달 ESC 확장
- └─ UIToast # 알림
- └─ UISystem # 시스템 화면

UI의 표시 순서와 공통 수명을 관리하는 구조 · 계속

개별 화면은 용도에 맞는 베이스를 상속한다. 장면 전환 관리나 게임 규칙을 베이스에 넣지 않고, 화면의 공통 동작과 확장 지점만 제공한다.

공통 실행 순서에 Template Method를 적용했다

TEXT

UIManager의 열기 요청
 → UIBase의 열림 상태 갱신
 → GameObject 활성화와 공통 처리
 → 해당 화면의 OnOpened 실행

UIBase는 실행 순서를 유지하고, 개별 화면은 초기화·열기·닫기 혹은 재정의한다. **공통 처리의 골격은 베이스에 두고 화면별 처리는 흑으로 확장하는 Template Method 구조다.**

중복 초기화는 거절하고, 이미 열린 화면에 다시 열기 요청이 와도 같은 동작을 반복하지 않도록 처리한다. enum 멤버 이름으로 자식 객체를 찾아 타입별 바인딩 배열에 보관하는 기능도 공통 베이스에 뒀다.

UIBase가 제공하는 API

API	사용 주체와 역할
Layer	화면 클래스가 자신의 표시 계층을 지정
IsInitialized, IsOpen	초기화·열림 상태 조회
RectTransform	초기화 시 보관한 화면 Transform 조회
OnInitialize()	개별 화면이 재정의해 바인딩·버튼 연결 등 초기 동작 구현
OnOpened()	개별 화면이 재정의해 열릴 때 데이터 갱신 등 구현
OnClosed()	개별 화면이 재정의해 닫힐 때 정리 동작 구현
Bind<T>(Type enumType)	enum 이름으로 자식 객체·컴포넌트를 찾아 타입별 배열에 보관
Get<T>(int index)	보관한 바인딩을 인덱스로 조회
GetObject, GetText, GetButton, GetImage	자주 쓰는 바인딩 타입의 조회 도우미
Initialize(), OpenFromManager(), CloseFromManager()	관리자가 호출하는 내부 수명 진입점

UIManager를 통한 열기·닫기는 화면 상태와 순서 목록을 함께 반영한다. 개별 화면의 흑은 그 공통 순서 안에서 실행된다.

UI의 표시 순서와 공통 수명을 관리하는 구조 · 계속

UIManager가 제공하는 API

API	역할
<code>RegisterAll(List<UIBase>)</code>	화면 목록 초기화, Root 배치, 타입별 등록
<code>Get<T>()</code>	등록된 화면을 타입으로 조회
<code>Open<T>(), Close<T>()</code>	화면 열기·닫기와 순서 목록 갱신
<code>TopWindow, TopPopup, TopModal</code>	해당 계층의 최상단 화면 조회
<code>PopupCount</code>	열린 팝업 수 조회
<code>CloseAllPopups()</code>	열린 팝업 전체 닫기
<code>Unregister(UIBase)</code>	화면 등록과 순서 목록·포커스 구독 정리
<code>Clear()</code>	등록 화면 제거와 관리 목록 초기화

C#

```
// 등록된 설정 화면을 타입으로 조회·열기·닫기
uiManager.Get<UI_Settings>();
uiManager.Open<UI_Settings>();
uiManager.Close<UI_Settings>();
```

ESC 입력의 처리 우선순위를 정했다

ESC 처리는 공통 경로에서 최상단 화면에 입력을 전달하고, 개별 화면이 기본 함수를 재정의하도록 제안했다. 해당 구현은 팀원이 반영했다.

여러 화면이 ESC를 각각 읽으면 같은 입력에 여러 화면이 반응할 수 있다. `GameInstance`의 공통 입력 경로에서 텍스트 입력 여부를 확인하고 닫기 우선순위를 적용한다.

TEXT

ESC 입력

- 텍스트 입력 중이면 건너뛴
- 별도 ESC 처리자 확인
- 최상단 Modal
- 최상단 Window

씬 진입점은 해당 씬의 UI를 등록하고, 씬 종료 시 등록을 해제한다. 화면 내용은 기능 서비스의 상태를 받아 표시한다. 네트워크가 화면 객체를 직접 선택하는 대신 서비스의 상태 변경을 화면이 구독한다.

UI의 표시 순서와 공통 수명을 관리하는 구조 · 계속

화면마다 ESC 동작을 재정의했다

UIWindow와 UIModal은 기본 `OnEscape()`를 제공한다. 기본 구현은 아무 동작도 하지 않으며, **개별 화면이 함수를 오버라이드해서 자신의 뒤로 가기·취소·재개 동작을 구현한다**. ESC 입력 자체는 공통 경로가 읽고 선택한 최상단 화면에 전달한다.

화면	재정의한 ESC 동작
방 생성	뒤로 가기 동작 호출
비밀방 비밀번호 모달	취소 동작 호출
게임 메뉴	플레이 재개 요청 발행
설정	드롭다운 닫기 → 키 재설정 취소 → 목록 포커스 복귀 또는 뒤로 가기

설정 화면은 ESC를 받았다고 바로 전체 화면을 닫지 않는다. 화면 내부에서 진행 중인 동작을 먼저 정리한다. **입력 전달 대상의 선택은 공통 경로에, ESC의 구체적인 의미는 개별 화면에 둔 구조다**.

enum 이름으로 자식을 찾아 참조한다

UIBase에는 enum 기반 이름 바인딩을 넣었다. 개별 화면이 버튼·텍스트·입력 필드의 이름을 enum으로 정의하면, 공통 바인딩 함수가 같은 이름의 자식 객체에서 필요한 타입의 참조를 찾는다.

TEXT

- 화면에서 enum과 컴포넌트 타입 지정
 - enum 멤버 이름 목록 조회
 - 비활성 객체를 포함한 하위 계층 탐색
 - 이름이 같은 객체의 컴포넌트 참조 선택
 - 타입별 배열에 보관
 - enum 인덱스로 저장된 참조 조회

비밀번호 모달에서 사용하는 실제 바인딩은 다음과 같다.

CSHARP

```
private enum Buttons
{
    PasswordSubmitButton,
    PasswordCancelButton
}

// 초기화에서 enum 이름에 맞는 Button 참조를 보관
Bind<Button>(typeof(Buttons));

// 이후에는 이름으로 다시 탐색하지 않고 저장된 참조를 조회
_submitButton = GetButton((int)Buttons.PasswordSubmitButton);
_cancelButton = GetButton((int)Buttons.PasswordCancelButton);
```

UI의 표시 순서와 공통 수명을 관리하는 구조 · 계속

텍스트와 입력 필드도 같은 방식으로 연결한다. 일반 조회 API가 있으므로 TextMeshPro와 TMP 입력 필드도 타입을 지정해 바인딩한다.

C#

```
Bind<TextMeshProUGUI>(typeof(Texts));
Bind<TMP_InputField>(typeof(InputFields));

_roomLabel = Get<TextMeshProUGUI>((int)Texts.PasswordRoomLabel);
_passwordInput = Get<TMP_InputField>((int)InputFields.PasswordInput);
```

enum 이름과 UI 객체 이름을 맞추는 것이 연결 규칙이다. 타입별로 바인딩 배열을 보관하므로 각 컴포넌트 타입에 대응하는 enum을 사용한다. enum 값은 배열 인덱스로 사용하므로 0부터 연속된 값으로 정의한다.

찾지 못한 객체는 경고를 남긴다. 개별 화면은 필수 참조를 검사해 잘못된 이름이나 누락된 연결을 초기화 단계에서 확인한다. 바인딩된 객체를 사용할 때마다 계층을 다시 탐색하지 않고 저장한 참조를 사용한다.

재사용할 공통 동작도 베이스에 모았다

기능	구성
드래그 가능한 창	UIWindow의 포커스 정책을 상속하고 UIDraggable에서 드래그 이동 구현
드래그 좌표	포인터 이동량을 Canvas 배율로 보정해 RectTransform에 반영
팝업 닫기 정책	수동·자동·자동 또는 입력 모드와 유지 시간 데이터 제공
중복 등록 검사	같은 타입의 살아 있는 화면이 이미 등록되면 거절
파괴된 화면 참조 정리	등록 Dictionary와 Window-Popup-Modal 순서 목록에서 함께 제거
등록 해제	화면 닫기, 표시 순서 갱신, 창 포커스 구독 해제

팝업 베이스는 닫기 정책 데이터를 제공한다. 해당 데이터 정의와 실제 타이머 실행은 별도 책임이다.

음성 채팅 UI도 서비스 상태를 받아 표시한다

Vivox 연동에서는 방의 음성 채팅 화면을 음성 서비스와 연결했다. 화면은 연결 상태와 참가자 변경 이벤트를 구독하고, 참가자별 연결·발언·음소거 상태를 표시한다. 발언 중인 참가자는 행의 시각적 강조로 구분한다.

TEXT

```
Vivox 참가자 이벤트
→ SDK Gateway에서 게임용 상태로 변환
→ 음성 서비스의 참가자 상태 갱신
→ 참가자 변경 이벤트
→ 방 음성 UI 갱신
```

화면은 SDK 참가자 객체를 직접 다루지 않고 게임용 상태 객체를 사용한다. 마이크 음소거는 서비스에 요청하며, 화면 종료 시 음성 상태와 참가자 이벤트 구독을 해제한다. **음성 연결의 수명은 서비스가, 화면 표시와 구독의 수명은 UI가 관리한다.**

UI의 표시 순서와 공통 수명을 관리하는 구조 · 계속

책임 분리

구성	책임
UIManager	화면 등록·조회, Layer 배치, 표시 순서
UIRoot	표시 계층의 부모 객체 참조
UIBase	공통 수명 순서, 상태, 바인딩
개별 화면	데이터 표시와 화면별 입력 동작
씬 진입점	씬의 화면 등록과 정리

표시 관리와 화면별 내용을 나눠 **새 화면이 공통 열기·닫기 규칙을 재사용하도록** 구성했다.

13 / Client Framework

캐릭터 행동·애니메이션·카메라의 변경 지점을 분리한 구조

캐릭터의 접지·공중·공격 행동은 상태로 나누고 장비와 카메라는 전략으로 교체했다. 게임 상태를 Animator 파라미터로 바꾸는 처리는 어댑터로 분리했다.

[블로그 원문 ↗](#)

상황·장비·시점에 따라 달라지는 동작을 나눴다

접지와 공중에서는 입력 처리 방식이 다르고, 장비가 바뀌면 공격과 애니메이션 구성이 달라진다. 카메라 시점마다 위치 계산도 다르다. 캐릭터의 상황은 상태로, 교체할 행동 정책은 전략으로 분리했다.

디렉터리와 파일 구조

TEXT

```
Client/Assets/Game/
├─ Core/StateMachines/
│   ├── ICharacterState.cs
│   └─ CharacterStateMachine.cs
└─ Features/
    ├── Player/
    │   ├── Config/
    │   │   └─ CharacterModeCatalog.cs
    │   └─ Runtime/
    │       ├── PlayerCameraController.cs
    │       ├── PlayerAnimatorDriver.cs
    │       ├── PlayerUpperBodyAnimator.cs
    │       ├── States/
    │       │   ├── PlayerStateContext.cs
    │       │   ├── PlayerGroundedState.cs
    │       │   ├── PlayerAirborneState.cs
    │       │   └─ PlayerBasicAttackState.cs
    │       ├── CharacterModes/
    │       │   ├── ICharacterModeStrategy.cs
    │       │   ├── CharacterModeSelector.cs
    │       │   ├── CharacterModeStrategies.cs
    │       │   └─ CharacterModeContext.cs
    │       └─ CameraStrategies/
    │           └─ ICameraViewStrategy.cs
```

TEXT · 계속

```
    ├── CameraStrategyContext.cs
    ├── FirstPersonCameraStrategy.cs
    ├── ThirdPersonCameraStrategy.cs
    └─ AimCameraStrategy.cs
└─ PlayerMovement/Runtime/
    └─ RemoteAnimatorDriver.cs
```

캐릭터 행동·애니메이션·카메라의 변경 지점을 분리한 구조 · 계속

캐릭터 행동에는 상태 패턴을 적용했다

상태 머신은 현재 상태를 보관하고 진입·갱신·종료 순서를 관리한다. 상태는 해당 상황에서 가능한 행동과 전환 조건을 판단한다.

TEXT

현재 상태의 Tick

- 상태별 입력과 전환 조건 처리
- 기존 상태 Exit
- 다음 상태 Enter

상태	책임
접지	지상 이동, 공격 진입, 점프와 공중 전환
공중	공중 행동과 상태 전환
기본 공격	공격 상태의 행동과 전환

접지 상태가 점프 입력을 받으면 Motor에 점프를 적용하고 애니메이션에 점프를 알린 뒤 공중 상태로 전환한다. **상태는 행동을 결정하고 Motor는 이동을 수행한다.**

장비 전략이 행동과 Animator 구성을 교체한다

장비 선택기는 아이템에 맞는 전략을 선택한다. 기본 전략은 맨손의 컨트롤러를 복원하고, 설정 기반 전략은 Catalog의 장비 정의를 사용한다.

TEXT

아이템 변경

- 장비 전략 선택
- 기존 전략 종료 / 새 전략 진입
- 캐릭터에 맞는 장비용 Animator Controller 선택
- 컨트롤러와 공격 설정 갱신

Catalog에는 장비 모드, 기본 공격 상태, 캐릭터별 컨트롤러 변형, 손 부착 위치를 정의했다. Bat/Gun마다 클래스를 늘리는 대신 공통 설정 기반 전략이 장비 데이터를 받아 실행한다.

전략은 주 행동과 장비 부착 정책을 제공하고 공격 레이어의 진입·종료도 관리한다. 컨트롤러가 달라지면 Context가 Animator를 다시 바인딩하고 이동·공격 및 상체 드라이버의 설정을 갱신한다.

장비 전략은 캐릭터 상태 계약도 사용한다. **장비별 행동 선택은 전략의 책임이고, 모드 전환의 진입·종료 순서는 상태 머신을 재사용한다.**

캐릭터 행동·애니메이션·카메라의 변경 지점을 분리한 구조 · 계속

애니메이션 드라이버는 게임 상태를 변환한다

TEXT

로컬 행동 상태 → 애니메이션 드라이버 → Animator
원격 확정 상태 → 원격 드라이버 → 공통 애니메이션 드라이버 → Animator

애니메이션 드라이버는 이동·점프·접지·공격을 Animator 파라미터와 트리거로 변환하는 어댑터다. 상태 클래스가 파라미터 문자열과 레이어 구성을 직접 다루지 않도록 경계를 뒀다.

원격 드라이버도 같은 애니메이션 규칙을 사용한다. 원격 이동 정보는 로컬 입력 크기의 규칙에 맞게 변환하고 달리기 여부는 별도로 전달한다. 장비 전략이 컨트롤러 구성을 선택하면 드라이버가 그 설정에 맞춰 게임 상태를 반영한다.

카메라 계산에는 전략 패턴을 적용했다

TEXT

```
PlayerCameraController
→ ICameraViewStrategy
  ├── FirstPersonCameraStrategy
  ├── ThirdPersonCameraStrategy
  └── AimCameraStrategy
```

카메라 제어기는 사용할 전략을 선택하고, 전략은 시점별 배치 계산을 맡는다. 모드를 바꿀 때 제어기는 기존 전략을 종료하고 새 전략에 진입한다. 같은 모드의 중복 전환은 건너뛴다.

조준 전략은 3인칭 전략을 내부에서 사용해 카메라 배치·충돌·완화 계산을 재사용한다. 조준 차이는 전략에 두고 공통 계산은 객체 합성으로 공유한다.

변경할 책임을 구분했다

변경	담당
캐릭터 상황별 행동과 전환	캐릭터 상태
장비별 공격과 컨트롤러 구성	장비 전략과 Catalog
게임 상태의 Animator 표현	애니메이션 드라이버
시점별 카메라 계산	카메라 전략

행동 결정, 장비 정책, 애니메이션 변환, 카메라 계산의 변경 지점을 나눈 구조다. 게임 페이지는 별도의 전략 객체로 구성하지 않고 진행 상태의 매핑과 전환 조정으로 연결했다.